



Instituto Tecnológico
de Buenos Aires

mPES: Esquemas para Toma de Decisión Artificial en Escenarios Secuenciales

*Una evaluación comparativa de arquitecturas de
Aprendizaje por Refuerzo para la asignación de recursos limitados
bajo incertidumbre utilizando como escenario de prueba una crisis
pandémica*

Alumno: Ing. Maximiliano Leonel Vega

Director: Dr. Ing. Rodrigo Ramele

Tesis de Maestría

Maestría en Ciencia de Datos

Instituto Tecnológico de Buenos Aires

2026

Resumen

Esta tesis compara métodos de aprendizaje por refuerzo en un problema de asignación secuencial con recursos limitados. Como escenario de prueba se emplea una pandemia simulada, en la cual las ciudades se presentan sucesivamente, cada una con una severidad inicial, y un agente decide cuántos recursos asignarle a partir de un presupuesto fijo. La severidad no atendida crece en cada paso, por lo que una asignación insuficiente en una ciudad se amplía en los pasos siguientes.

Se entrenaron seis agentes con hiperparámetros optimizados: dos tabulares (Q-Learning y Double Q-Learning) y cuatro redes neuronales (una red Q profunda, una variante recurrente que procesa los últimos estados, un método actor-crítico y un Transformer causal que atiende a la historia reciente), más un Q-Learning sin optimizar como referencia. Se evaluaron además seis reglas para combinar las redes en un ensamble, en las que cada red se pondera según su confianza en la decisión, derivada de la entropía de Shannon. Todos los modelos se evaluaron sobre las mismas 64 secuencias de ciudades en las condiciones de entrenamiento y en 21 escenarios con otras severidades o longitudes, tres de ellos fuera del rango de entrenamiento.

Como métrica de comparación se emplea el desempeño normalizado de cada secuencia, donde 0 representa la ausencia de asignación y 1 la asignación óptima, calculada con conocimiento de la secuencia completa. El análisis identifica al Transformer como el mejor modelo individual, tanto en las condiciones de entrenamiento como en los escenarios de generalización.

Entre los ensambles, sólo el ensamble ponderado supera al Transformer individual, por un margen reducido, y aquel que incluye una compuerta que regula la participación del Transformer alcanza un desempeño equivalente, porque ejecuta la acción de este en casi todas las decisiones. La ventaja del ensamble ponderado se asocia a una temperatura alta y a un *prior* de severidad más que a la ponderación por confianza. Los ensambles que sólo promedian o votan con pesos derivados de la entropía de Shannon quedan por debajo del Transformer individual, por lo que esa confianza no basta por sí sola para superar al mejor miembro en términos de desempeño promedio. Una regla fija que asigna la severidad más dos unidades, calibrada con la asignación óptima, alcanza el desempeño del mejor ensamble, lo que indica que la política óptima de este entorno tiene una estructura simple.

Palabras clave: aprendizaje por refuerzo, asignación secuencial con recursos limitados, Transformer causal, ensambles de modelos, entropía de Shannon, generalización.

Abstract

This thesis compares reinforcement learning methods on a sequential allocation problem with limited resources. A simulated pandemic is used as the test scenario: cities are presented sequentially, each with an initial severity, and an agent decides how many resources to allocate to each one from a fixed budget. Unaddressed severity grows at every step, so an insufficient allocation to a city is amplified in the following steps.

Six agents with optimised hyperparameters were trained: two tabular methods (Q-Learning and Double Q-Learning) and four neural networks (a deep Q-network, a recurrent variant that processes the most recent states, an actor-critic method and a causal Transformer that attends to the recent history), plus a non-optimised Q-Learning as a reference. Six rules for combining the networks into an ensemble were also evaluated; in all of them each network is weighted by its confidence in the decision, derived from Shannon entropy. All models were evaluated on the same 64 sequences of cities under the training conditions and in 21 scenarios with other severities or lengths, three of them outside the training range.

The comparison metric is the normalised performance of each sequence, where 0 represents no allocation and 1 the optimal allocation, computed with knowledge of the full sequence. The analysis identifies the Transformer as the best individual model, both under training conditions and in the generalisation scenarios.

Among the ensembles, only the weighted ensemble outperforms the individual Transformer, by a small margin, and the one that includes a gate regulating the participation of the Transformer reaches an equivalent performance, since it executes the Transformer's action in almost every decision. The advantage of the weighted ensemble is associated with a high temperature and a severity prior rather than with confidence weighting. The ensembles that only average or vote with weights derived from Shannon entropy fall below the individual Transformer, so this confidence is not sufficient on its own to outperform the best member in terms of average performance. A fixed rule that allocates the severity plus two units, calibrated with the optimal allocation, reaches the performance of the best ensemble, which indicates that the optimal policy of this environment has a simple structure.

Keywords: reinforcement learning, sequential allocation with limited resources, causal Transformer, model ensembles, Shannon entropy, generalisation.

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Definición del problema	1
1.3. Preguntas e hipótesis	1
1.4. Contribuciones	2
2. Marco Teórico	3
2.1. Procesos de decisión de Markov	3
2.2. Aprendizaje por refuerzo basado en valores	4
2.3. Memoria, atención y actor-crítico	4
2.4. Entropía como medida de confianza	5
2.5. Ensamblés	5
2.6. Optimización bayesiana de hiperparámetros	6
2.7. Comparación estadística	6
3. Estado de la Cuestión	8
3.1. Aprendizaje por refuerzo para decisiones secuenciales bajo incertidumbre	8
3.2. Transformers en aprendizaje por refuerzo	9
3.3. Ensamblés en aprendizaje por refuerzo	9
3.4. Generalización bajo variaciones controladas	10
4. Materiales y Métodos	11
4.1. Implementación	11
4.2. Entorno	11
4.3. Métrica de desempeño	12
4.4. Modelos individuales	13
4.4.1. Modelos tabulares	14
4.4.2. Modelos con redes neuronales	15
4.5. Ensamblés	16
4.6. Optimización de hiperparámetros	19
4.7. Escenarios de generalización	19
4.8. Protocolo de evaluación	20
5. Resultados	21
5.1. Mejor modelo individual	21
5.2. Ensamblés	23
5.2.1. Efecto de las reglas fijas sobre la decisión	25
5.3. Transformer frente al mejor ensamble	28
5.4. Réplicas fuera de muestra	30
5.5. Justificación a posteriori del ensamble ponderado	31
6. Discusión	33
6.1. Interpretación de la ventaja del Transformer	33
6.2. Efecto de la regla de combinación	33
6.3. Alcance de los resultados frente a una regla fija	34

7. Conclusiones	36
7.1. Respuesta a las preguntas de investigación	36
7.2. Limitaciones del estudio	37
7.3. Trabajo futuro	37
7.4. Conclusión general	38
Referencias Bibliográficas	39
Apéndice	44
A. Comandos de reproducción	44
B. Ejecución de la evaluación	44
C. Mapas estadísticos complementarios	45
D. Resultados por secuencia de cada modelo	51

Índice de figuras

4.1. Agente aleatorio	14
5.1. Desempeño medio por escenario (modelos individuales)	22
5.2. Curvas por secuencia en seis escenarios (modelos individuales)	23
5.3. Desempeño medio por escenario (ensambles)	27
5.4. Curvas de los ensambles en los escenarios fuera de rango	29
5.5. Resultados por secuencia: ensamble ponderado	29
C.1. Welch por escenario (modelos individuales)	46
C.2. d de Cohen entre pares (modelos individuales)	47
C.3. Welch por escenario (ensambles)	48
C.4. d de Cohen por escenario (ensambles)	49
C.5. Comparación entre pares (ensambles)	50
D.1. Resultados por secuencia: Q-Learning base	51
D.2. Resultados por secuencia: Q-Learning	52
D.3. Resultados por secuencia: Double Q-Learning	52
D.4. Resultados por secuencia: DQN	53
D.5. Resultados por secuencia: DQN recurrente	53
D.6. Resultados por secuencia: A2C	54
D.7. Resultados por secuencia: Transformer	54

Índice de tablas

4.1. Modelos individuales	14
4.2. Hiperparámetros de los modelos tabulares	14
4.3. Hiperparámetros de los modelos de tipo DQN	16
4.4. Hiperparámetros de A2C	16
4.5. Parámetros de los ensambles	19
4.6. Escenarios de evaluación	20
5.1. Desempeño de los modelos individuales	21

5.2.	Desempeño de los ensambles	24
5.3.	Comparación de cada ensamble con el Transformer	24
5.4.	Frecuencia con que las reglas cambian la decisión	26
5.5.	Transformer frente al ensamble ponderado	28
5.6.	Referencia frente a las réplicas fuera de muestra	30
5.7.	Sensibilidad del ensamble ponderado	31
5.8.	Regla fija frente a los modelos	32
A.1.	Comandos de reproducción	44

1. Introducción

Esta tesis compara arquitecturas de aprendizaje por refuerzo (RL, *Reinforcement Learning*) en una simulación de pandemia donde el agente estudiado distribuye un presupuesto fijo entre ciudades que se presentan sucesivamente, sin conocer cuántas restan ni cuál será su severidad. El entorno, **mPES**, extiende el *Pandemic Experiment Scenario* (PES) del BCI-NE Lab de la Universidad de Essex [1].

1.1. Motivación

Durante la pandemia de COVID-19, gobiernos y hospitales debieron asignar recursos escasos en condiciones en que cada asignación modificaba la evolución posterior del sistema sanitario. En esta tesis la pandemia no es el objeto de estudio, sino un escenario controlado que permite medir qué modelos toman mejores decisiones y cómo se comportan fuera de las condiciones de entrenamiento.

1.2. Definición del problema

En cada paso se presenta una ciudad y el agente le asigna entre 0 y 10 unidades de recurso. La severidad de cada ciudad crece en un factor β por paso y cada unidad asignada la reduce en una cantidad α (Ecuación (4.3)). Esta regla simplifica los modelos de la epidemiología computacional [2], desde los compartimentales hasta simulaciones espaciales de COVID-19 basadas en agentes, como las de Parisi *et al.* [3]: conserva sólo el crecimiento de la severidad y el efecto de los recursos.

1.3. Preguntas e hipótesis

1. ¿Qué modelo individual obtiene el mayor desempeño, y lo conserva cuando cambian la severidad o la longitud de las secuencias?
2. ¿Algún ensamble de modelos ya entrenados supera al mejor modelo individual y, en tal caso, con qué regla de combinación?

Para cada pregunta se plantea una hipótesis:

H1. *Entre los modelos individuales evaluados, el Transformer causal alcanza el mayor desempeño medio y lo conserva en los escenarios de generalización.*

H2. *Los ensambles de redes neuronales con votación basada en la confianza derivada de la entropía de Shannon superan el desempeño del mejor modelo individual.*

1.4. Contribuciones

1. Una comparación de seis modelos de RL, evaluados con los mismos entorno, protocolo y cantidad de secuencias, con hiperparámetros de entrenamiento ajustados por optimización bayesiana.
2. Seis reglas para combinar esos modelos sin reentrenarlos, en las que cada modelo se pondera según la entropía de su distribución de acciones.
3. Una evaluación en 21 escenarios que cambian la severidad, la longitud de las secuencias o ambas, en un *framework* reproducible.

2. Marco Teórico

Este capítulo formaliza el problema y resume los métodos empleados en la tesis; la referencia general es Sutton y Barto [4].

2.1. Procesos de decisión de Markov

Un proceso de decisión de Markov (MDP) finito es una tupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$, con un conjunto finito de estados $\mathcal{S}$, un conjunto finito de acciones $\mathcal{A}$, probabilidades de transición $P(s' | s, a)$, una función de recompensa r y un factor de descuento $\gamma \in [0, 1)$. Su rasgo definitorio es la propiedad de Markov: el estado siguiente depende sólo del estado y de la acción actuales.

$$P(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0) = P(s_{t+1} | s_t, a_t) \quad (2.1)$$

Donde s_t y a_t son el estado y la acción en el paso t . El agente busca la política $\pi(a | s)$ que maximiza el retorno descontado esperado:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1} \quad (2.2)$$

Donde r_{t+k+1} es la recompensa recibida $k + 1$ pasos después de t y T el último paso del episodio. El valor de una acción bajo la política π es $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$, y el de la política óptima satisface la ecuación de Bellman:

$$Q^*(s, a) = \mathbb{E} \left[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a \right] \quad (2.3)$$

Donde la esperanza se toma sobre el estado siguiente y la recompensa. La política óptima elige en cada estado $\arg \max_a Q^*(s, a)$. Si la dinámica y toda la secuencia se conocen de antemano, la Ecuación (2.3) se resuelve en forma exacta por programación dinámica [4]; en esta tesis, ese óptimo con información completa es la referencia superior de la métrica (Sección 4.3).

Cuando el agente no observa todas las variables de las que dependen la recompensa y

la evolución futura, el problema es un MDP parcialmente observable (POMDP) [5]: la observación actual no resume la historia, y las observaciones pasadas pueden aportar información útil para decidir.

2.2. Aprendizaje por refuerzo basado en valores

Q-Learning. Estima Q^* en una tabla [6] con la actualización $Q(s, a) \leftarrow Q(s, a) + \alpha_{lr} [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$, con tasa de aprendizaje α_{lr} , y explora con una política ε -greedy, que elige una acción al azar con probabilidad ε y la de mayor Q en otro caso; ε decrece de ε_0 a $\varepsilon_{\min}$ durante el entrenamiento para pasar de explorar a explotar lo aprendido.

Double Q-Learning. Mantiene dos tablas [7], una para elegir la acción siguiente y otra para evaluarla, con lo que reduce la sobreestimación que introduce el máximo.

Deep Q-Network (DQN). Aproxima Q con una red neuronal [8] y estabiliza el entrenamiento con un *buffer* de repetición de experiencia (*experience replay*) [9] y una red objetivo (*target network*) que se copia cada C pasos. Double DQN [10] elige la acción siguiente con la red en línea y la evalúa con la red objetivo. Los tres agentes de tipo DQN de esta tesis usan ese valor objetivo y enmascaran las acciones no factibles, que nunca pueden resultar elegidas. Se entrenan con Adam [11], con pesos iniciales de Glorot [12], que mantienen la escala de las activaciones entre capas, y con dos recursos que acotan cada actualización: la pérdida de Huber [13], cuadrática para errores pequeños y lineal para los grandes, y el recorte de la norma del gradiente [14]. El aprendizaje empieza cuando el *buffer* contiene suficientes transiciones para que los primeros lotes no provengan de unos pocos episodios [8].

Modelado de recompensa por potencial (PBRs). Suma a la recompensa el término $\kappa[\gamma\Phi(s') - \Phi(s)]$ [15], que no cambia las políticas óptimas y puede acelerar el aprendizaje cuando la recompensa informa poco sobre el progreso; en esta tesis, $\Phi(s) = -\sum_i S_i$ es la suma de las severidades actuales con signo negativo (Sección 4.2).

2.3. Memoria, atención y actor-crítico

Cuando el estado observado no contiene toda la información relevante (POMDP), puede procesarse una ventana de estados recientes. La LSTM [16] lo hace con un estado

oculto y compuertas de entrada, olvido y salida que regulan qué información se conserva; Hausknecht y Stone [17] mostraron que un DQN con una LSTM iguala al DQN con cuadros apilados en entornos parcialmente observables y se degrada menos cuando cambia la calidad de las observaciones. El Transformer [18] reemplaza la recurrencia por atención de producto escalar: cada posición combina las demás con pesos que dependen de su similitud, en varias cabezas en paralelo, y un vector de posición sumado a la entrada indica el orden. Cada bloque alterna atención y una capa *feed-forward*, con conexiones residuales y normalización de capa; con una máscara causal, cada posición sólo atiende a las anteriores. Los métodos actor-crítico derivados del gradiente de política [19], como A2C, la variante síncrona de A3C [20], aprenden a la vez una política y una función de valor; la ventaja se estima con GAE [21], que promedia estimaciones de varios pasos con un factor λ , y un término de entropía con coeficiente β_H evita la convergencia prematura de la política.

2.4. Entropía como medida de confianza

La entropía de Shannon [22] de una distribución p sobre las acciones es:

$$H(p) = - \sum_{a \in \mathcal{A}} p(a) \log p(a) \quad (2.4)$$

Donde $p(a)$ es la probabilidad que el modelo asigna a la acción a . La entropía vale 0 si toda la probabilidad está en una acción y alcanza su máximo, $\log |\mathcal{A}|$, con la distribución uniforme. La entropía de la distribución predictiva es una medida habitual de la incertidumbre de un modelo [23]; normalizada por ese máximo, queda en $[0, 1]$. En esta tesis $1 - H(p)/\log |\mathcal{A}|$ se llama *confianza* (Ecuación (4.8)): vale 1 si toda la probabilidad está en una acción y 0 si la distribución es uniforme. Los agentes de tipo DQN producen valores Q , no probabilidades, por lo que la confianza se calcula sobre una distribución derivada de ellos (Sección 4.5) y no es una probabilidad calibrada.

2.5. Ensamblajes

Un ensamble combina las salidas de varios modelos ya entrenados. Los valores Q de cada miembro se convierten en una distribución con una *softmax* de temperatura τ

(Ecuación (4.7)): un τ pequeño concentra la probabilidad en la mejor acción y uno grande la acerca a la uniforme. En el voto suave (*soft voting*) se promedian las distribuciones con pesos w_k y se elige la acción más probable; en el voto duro (*hard voting*), que en esta tesis se llama voto por acción, cada miembro vota por su acción preferida [24]. Los pesos pueden depender de la confianza de cada miembro en la decisión actual. Una compuerta, como en la mezcla de expertos [25], decide según la entrada qué miembro determina la salida.

Las reglas de combinación pueden incorporar también conocimiento del dominio que no proviene de los miembros: mezclar la distribución del ensamble con una distribución previa (*prior*) sobre las acciones, o reemplazar las acciones que violan una restricción, como el blindaje (*shielding*) del RL seguro [26]. El *prior* de severidad y la cota de seguridad de la Sección 4.5 son reglas de este tipo, diseñadas para este problema: asignan aproximadamente según la severidad y evitan asignaciones muy bajas a ciudades graves.

2.6. Optimización bayesiana de hiperparámetros

Los hiperparámetros se ajustan con el estimador estructurado en árbol de Parzen (TPE) [27] de Optuna [28], que modela por separado la densidad de las configuraciones de mayor y de menor puntaje y propone configuraciones donde el cociente entre ambas es alto. Los ensayos poco prometedores se interrumpen con una regla de poda (*pruning*) que compara su puntaje intermedio con la mediana de los ensayos anteriores. En este trabajo, los hiperparámetros se optimizaron con el objetivo de maximizar el desempeño promedio en las secuencias de referencia. Como se elige la configuración de mayor puntaje sobre esas mismas secuencias, ese puntaje tiende a sobreestimar el desempeño en secuencias nuevas (sesgo de selección [29]); por eso el desempeño se mide también en réplicas independientes de la misma distribución (Sección 5.4).

2.7. Comparación estadística

Las diferencias entre modelos se expresan con la d de Cohen [30], la diferencia de medias en desviaciones estándar, y se leen como efecto despreciable ($|d| < 0,2$), pequeño (0,2 a 0,5), medio (0,5 a 0,8) o grande ($\geq 0,8$). La prueba t de Welch [31] indica si la diferencia es significativa. Su valor p se informa por el orden de magnitud: $p < 10^{-k}$ equivale a

$\log_{10} p < -k$, y cuanto más negativo es $\log_{10} p$, menos compatible es la diferencia observada con la igualdad de medias. Como cada comparación junta más de 1 300 secuencias por modelo, diferencias pequeñas producen valores p muy bajos: p indica si hay diferencia y d , cuán grande es. Además, todos los modelos enfrentan las mismas secuencias y no se corrige por comparaciones múltiples, por lo que los valores p son orientativos y la medida principal es la d . La prueba de Welch se usa en las comparaciones que juntan escenarios o grupos evaluados por separado. Cuando dos modelos se comparan secuencia por secuencia, se usa la prueba t pareada [32], que analiza las diferencias entre ambos en cada secuencia, y la concordancia entre dos ordenamientos se mide con la correlación de rangos de Spearman [33], r_s .

3. Estado de la Cuestión

El entorno de partida, PES [1], fue desarrollado por el BCI-NE Lab de la Universidad de Essex para estudiar decisiones tomadas en conjunto por personas y agentes artificiales [34, 35]. Esta tesis utiliza únicamente el entorno; todas las decisiones son tomadas por agentes artificiales.

3.1. Aprendizaje por refuerzo para decisiones secuenciales bajo incertidumbre

Las decisiones secuenciales bajo incertidumbre se formalizan como procesos de decisión de Markov o, cuando el agente no observa todo el estado, como POMDP [5]. Kochenderfer [36] y Powell [37] tratan el RL como una de varias familias de métodos para estos problemas, junto con la programación dinámica y la optimización estocástica.

Una primera línea aplica RL a la respuesta ante epidemias. Libin *et al.* [38] entrenaron un agente de gradiente de política (PPO, *Proximal Policy Optimization*) en un modelo metapoblacional de gripe pandémica con 379 distritos de Gran Bretaña; en un distrito contrastaron la política aprendida con una referencia conocida, que no pudieron establecer para el problema conjunto de 11 distritos. Ohi *et al.* [39] entrenaron un Double DQN con capas LSTM para fijar restricciones de movilidad en un escenario inspirado en COVID-19, equilibrando contagios y actividad económica. Bastani *et al.* [40] desplegaron en las fronteras de Grecia un sistema de RL que asignaba una cantidad limitada de tests e identificó 1,85 veces más viajeros infectados asintomáticos que el muestreo aleatorio, y Bednarski *et al.* [41] usaron RL para redistribuir equipamiento médico escaso entre estados de EE. UU.

Una segunda línea trata la asignación secuencial de recursos en salud y en operaciones. Komorowski *et al.* [42] aprendieron, a partir de registros de terapia intensiva, políticas de dosificación para pacientes con sepsis, y Yu *et al.* [43] revisan aplicaciones similares en salud. DeepRM [44] aprendió a asignar tareas con demandas de varios recursos con un desempeño comparable al de heurísticas establecidas; Gijssbrechts *et al.* [45] obtuvieron con A3C resultados equiparables a heurísticas y a programación dinámica aproximada en

gestión de inventarios, y Kong *et al.* [46] mostraron que agentes de RL aprenden, en la mochila en línea (*online knapsack*: cada ítem se acepta o se rechaza al llegar, sin conocer los siguientes y con capacidad limitada), comportamientos consistentes con los algoritmos óptimos conocidos. Dulac-Arnold *et al.* [47] formalizan los obstáculos que dificultan llevar estos métodos a sistemas reales.

mPES comparte con estos trabajos la asignación secuencial de un recurso escaso sin conocer la demanda futura, en una forma cercana a la mochila en línea. Se diferencia en tres aspectos: el entorno es controlado y de bajo costo, el presupuesto es fijo por secuencia y existe un óptimo exacto con información completa, calculado por programación dinámica, contra el cual se normaliza el desempeño de cada secuencia. Además, en lugar de ajustar una política para un sistema real, compara arquitecturas y ensambles sobre los mismos escenarios.

3.2. Transformers en aprendizaje por refuerzo

En RL, el Transformer se ha empleado de dos maneras. El *Decision Transformer* [48] aprende de trayectorias ya registradas y genera acciones condicionadas al retorno deseado. La segunda lo emplea como codificador de la historia en un agente que aprende interactuando con el entorno: Parisotto *et al.* [49] mostraron que en ese caso el Transformer estándar es difícil de entrenar y que, con normalización de capa antes de cada subcapa y compuertas en las conexiones residuales, alcanza o supera el desempeño de las LSTM en tareas que requieren memoria. El Transformer de esta tesis sigue el segundo enfoque: es un DQN cuyo codificador procesa los últimos estados. Como el estado de mPES no incluye la severidad de las ciudades ya atendidas ni la longitud de la secuencia (Sección 4.2), comparar modelos con ventana (DQN recurrente y Transformer) y sin ella (DQN) indica si esa información mejora las decisiones.

3.3. Ensamblajes en aprendizaje por refuerzo

Lakshminarayanan *et al.* [23] mostraron que promediar redes de la misma arquitectura, entrenadas desde inicializaciones distintas, mejora la estimación de la incertidumbre. Wiering y van Hasselt [24] combinaron las políticas de cinco algoritmos distintos, entre ellos Q-learning, Sarsa y actor-crítico, mediante votación y combinaciones de distribuciones

de Boltzmann. Los ensambles de esta tesis siguen esa línea, pero combinan modelos ya entrenados y ponderan a cada uno por la entropía de su distribución de acciones.

3.4. Generalización bajo variaciones controladas

Henderson *et al.* [50] encontraron que los algoritmos de RL profundo son sensibles a la semilla, a los hiperparámetros y a detalles de implementación; Cobbe *et al.* [51] mostraron que agentes con buen rendimiento en los niveles de entrenamiento no generalizaban a niveles nuevos producidos con la misma regla, y Kirk *et al.* [52] recomiendan evaluar sobre familias de variaciones controladas del entorno. Siguiendo esa recomendación, cada modelo se evalúa en 21 escenarios de generalización, tres de ellos por encima de las cotas del entorno (Sección 4.7).

4. Materiales y Métodos

4.1. Implementación

Los experimentos se implementaron en el repositorio `mPES` [53]. Cada paquete (`tabular`, `ml` y `ens`) incluye el entorno, implementado con Gymnasium [54], su entrenamiento y, cuando corresponde, su optimización bayesiana. Se utilizó Python 3.12 con TensorFlow 2.21, Keras 3.13, NumPy 2.4, Optuna 4.7, Gymnasium 1.2 y SciPy 1.17; los comandos de reproducción están en el Apéndice A.

4.2. Entorno

El entorno se formula como un MDP finito (Sección 2.1). En cada paso (*trial*) se presenta una ciudad y el agente decide cuántas unidades de un recurso escaso le asigna. Los estados y las acciones son:

$$\begin{aligned}\mathcal{S} &= \{(R, t, S) : R \in \{0, \dots, 30\}, t \in \{0, \dots, 10\}, S \in \{0, \dots, 9\}\} \\ \mathcal{A} &= \{0, 1, \dots, 10\}\end{aligned}\tag{4.1}$$

Donde R son los recursos restantes, t el paso dentro de la secuencia y S la severidad inicial de la ciudad actual; hay $31 \times 11 \times 10 = 3410$ estados. Cada secuencia dispone de 39 recursos, 9 de ellos preasignados por el entorno, por lo que el agente administra 30. Al elegir la acción a_t en el estado $s_t = (R_t, t, S_t)$, el estado pasa a:

$$s_{t+1} = (R_t - \min(a_t, R_t), t + 1, S_{t+1})\tag{4.2}$$

Donde $\min(a_t, R_t)$ es la asignación efectiva, porque una asignación mayor que los recursos disponibles se recorta, y S_{t+1} es la severidad inicial de la ciudad siguiente, que proviene de la secuencia y no depende de las acciones. La secuencia termina después de L pasos, con L desconocido para el agente. Como R y t evolucionan de forma determinista con la acción y S_{t+1} no depende de las decisiones, la transición se modela como markoviana (Ecuación (2.1)). La severidad de toda ciudad i ya incorporada evoluciona según:

$$S_{i,t+1} = \max(0, \beta S_{i,t} - \alpha a_i) \quad \alpha = 0,4 \quad \beta = 1 + \alpha = 1,4 \quad (4.3)$$

Donde a_i es la asignación que recibió la ciudad i al incorporarse, α la eficacia de cada unidad de recurso y β el factor de crecimiento por paso. Como la asignación se fija una sola vez y la severidad se actualiza en todos los pasos siguientes, una asignación insuficiente se amplifica geométricamente. La recompensa de cada paso es la suma, con signo negativo, de las severidades de las ciudades activas:

$$r_{t+1} = - \sum_{i \leq t} S_{i,t+1} \quad (4.4)$$

Donde la suma recorre las ciudades incorporadas hasta el paso t . Con esta recompensa, maximizar el retorno (Ecuación (2.2)) equivale a minimizar la severidad acumulada. Quedan así definidos los componentes del MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$: estados y acciones (Ecuación (4.1)), transición (Ecuaciones (4.2) y (4.3)) y recompensa (Ecuación (4.4)); el descuento γ es un hiperparámetro de cada agente. Sin embargo, la recompensa depende de las severidades de las ciudades ya atendidas y el final de la secuencia depende de L , y ninguno de los dos forma parte de (R, t, S) : para el agente, el problema es parcialmente observable (Sección 2.1). Los modelos con ventana de estados (Sección 4.4.2) procesan los últimos estados para recuperar parte de esa información.

Cada experimento tiene 8 bloques de 8 secuencias de 3 a 10 pasos (360 pasos en total). La referencia usa las 64 secuencias fijas de `initial_severity.csv` y `sequence_lengths.csv`, idénticas en todos los paquetes. El entrenamiento sortea longitudes y severidades de las distribuciones empíricas de esos mismos archivos.

4.3. Métrica de desempeño

Sea S_{agente} la suma de las severidades finales de las ciudades de una secuencia. Se normaliza con dos referencias: S_{peor} , la que resulta de no asignar nada, y S_{mejor} , la mínima alcanzable con el presupuesto conociendo de antemano toda la secuencia, que se calcula en forma exacta por programación dinámica sobre las asignaciones enteras de cada ciudad (Sección 2.1). El desempeño normalizado es:

$$\bar{r} = \frac{S_{\text{peor}} - S_{\text{agente}}}{S_{\text{peor}} - S_{\text{mejor}}} \quad (4.5)$$

Donde $\bar{r} = 0$ equivale a no asignar recursos y $\bar{r} = 1$ a la asignación óptima. Ambas referencias se calculan al terminar la secuencia y no intervienen en las decisiones.

Asignación óptima. Como la asignación de cada ciudad se fija al incorporarse y su severidad no depende de las demás ciudades, la severidad final de la ciudad c ($c = 0, \dots, L - 1$) depende sólo de su asignación a y de los $L - c$ pasos que evoluciona hasta el final: $f_c(a)$ es el resultado de aplicar $L - c$ veces la Ecuación (4.3) a su severidad inicial. S_{mejor} minimiza $\sum_c f_c(a_c)$ con asignaciones enteras $0 \leq a_c \leq 10$ que suman a lo sumo el presupuesto $B = 30$. Es un problema de mochila acotada que se resuelve en forma exacta por programación dinámica sobre las ciudades y los recursos restantes r :

$$V_c(r) = \min_{0 \leq a \leq \min(10, r)} [f_c(a) + V_{c+1}(r - a)] \quad V_L(r) = 0 \quad S_{\text{mejor}} = V_0(B) \quad (4.6)$$

Donde $V_c(r)$ es la mínima severidad final de las ciudades $c, \dots, L - 1$ con r recursos disponibles. La recursión recorre las ciudades desde la última hasta la primera y evalúa, para cada una, 11 acciones en cada uno de los 31 valores de r ; guardando la acción que alcanza cada mínimo se reconstruye la asignación óptima de cada ciudad, que se usa en la Sección 5.5. Es la forma finita y determinista de la ecuación de Bellman (Ecuación (2.3)), pero requiere conocer de antemano todas las severidades y la longitud L , información que el agente no tiene: ningún agente puede superar ese óptimo, y ninguno puede garantizar alcanzarlo.

Un agente que elige acciones al azar obtiene $\bar{r} = 0,670$ en las 64 secuencias de referencia (desviación estándar 0,186, rango [0,164; 0,950]; Figura 4.1).

4.4. Modelos individuales

La Tabla 4.1 lista los modelos, su paquete y el nombre que se usa en el texto; las figuras usan el nombre del paquete.

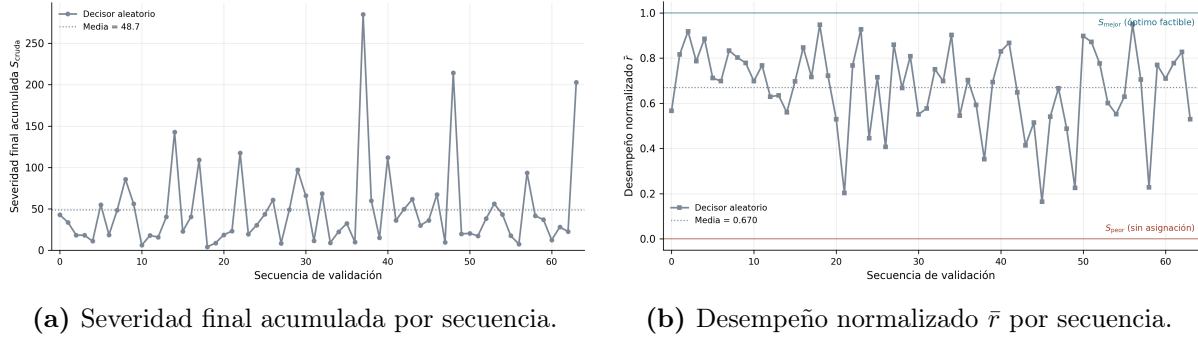


Figura 4.1: Agente aleatorio (*decisor aleatorio* en la figura) en las 64 secuencias de referencia; la línea punteada indica la media.

Tabla 4.1: Modelos individuales. Q-Learning base se incluye como referencia sin optimización.

Paquete	Nombre en el texto	Descripción
pes_base	Q-Learning base	Q-Learning tabular sin optimizar
pes_q1	Q-Learning	Q-Learning tabular optimizado
pes_dq1	Double Q-Learning	Double Q-Learning tabular con PBRS
pes_dqn	DQN	Double DQN con red densa
pes_rdqn	DQN recurrente	Double DQN con codificador LSTM
pes_a2c	A2C	Actor-crítico
pes_trf	Transformer	Double DQN con codificador Transformer causal

4.4.1. Modelos tabulares

Los modelos tabulares guardan Q en una tabla de $31 \times 11 \times 10 \times 11 = 37\,510$ valores inicializados con $\mathcal{U}(-1, 1)$ y enmascaran las acciones que exceden los recursos. La tabla sólo tiene filas para $S \leq 9$ y $t \leq 10$, por lo que una severidad mayor se recorta a 9 y un paso posterior, a 10. El modelo base usa los hiperparámetros originales de PES y Q-Learning, los de la optimización bayesiana; ambos reducen ε linealmente. Double Q-Learning mantiene ε_0 durante una fracción w de los episodios, la reduce exponencialmente hasta $\varepsilon_{\min}$ en la fracción q , usa PBRS (Sección 2.2) y decide con $\arg \max_a [Q_A(s, a) + Q_B(s, a)]$ (Tabla 4.2).

Tabla 4.2: Hiperparámetros de los modelos tabulares (Sección 2.2).

	Q-Learning base	Q-Learning	Double Q-Learning
Tasa de aprendizaje α_r	0,2	0,2862	0,1132
Descuento γ	0,9	0,8587	0,9777
$\varepsilon_0 / \varepsilon_{\min}$	0,8 / 0	0,6813 / 0,0437	0,5998 / 0,0327
Fracciones w / q	—	—	0,0260 / 0,6430
PBRS κ	—	—	0,000392
Episodios	1 000 000	550 000	360 000

4.4.2. Modelos con redes neuronales

Las redes reciben el estado escalado por las cotas del entorno, $[R/30, t/10, S/9]$; en los escenarios fuera de rango, las de tipo DQN reciben valores mayores que 1, y A2C recorta S a 9. Todas usan Adam [11], inicialización Glorot uniforme [12], recorte de la norma del gradiente y semilla $42 + i + 1$, con i el índice del mejor ensayo de su optimización. DQN, DQN recurrente y Transformer comparten el valor objetivo de Double DQN, la pérdida de Huber [13] (Sección 2.2), la exploración de Double Q-Learning y PBRS cuando $\kappa > 0$, y empiezan a aprender cuando el *buffer* alcanza una fracción f de su capacidad. Difieren en el procesamiento de la entrada; las tres terminan en una capa de salida de 11 valores Q :

- **DQN**: dos capas densas de 64 unidades sobre el estado actual.
- **DQN recurrente** [17]: una LSTM de 64 unidades recorre los últimos $W = 6$ estados, completados con ceros al inicio de cada secuencia, y su último estado oculto pasa por dos capas densas de 96 unidades. El estado oculto no se conserva entre decisiones.
- **Transformer**: la misma ventana, proyectada a $d_{\text{model}} = 32$ y sumada a un vector de posición fijo, pasa por 2 bloques con atención causal de 4 cabezas de dimensión 16 y una capa *feed-forward* de 64 unidades, sin *dropout*. Cada subcapa aplica la normalización de capa después de la conexión residual (*Post-LN*, como en [18]), sin las compuertas de Parisotto *et al.* [49]; la última posición pasa por una capa densa de 32 unidades. La atención no enmascara las filas de ceros que completan la ventana: sólo aplica la máscara causal.

La arquitectura del DQN recurrente y la del Transformer se eligieron mediante exploraciones *ad hoc*, debido al costo computacional de incluirlas en la búsqueda bayesiana; en ellos la optimización sólo ajustó los hiperparámetros de entrenamiento (Tabla 4.3).

A2C [20] (Sección 2.3) usa redes separadas con una capa oculta de 128 unidades para el actor, con salida *softmax* sobre las 11 acciones, y para el crítico. La ventaja se estima con GAE y se estandariza por lote; la política y su entropía se calculan sobre las acciones factibles (Tabla 4.4).

Tabla 4.3: Hiperparámetros de DQN, DQN recurrente y Transformer. Entre paréntesis, las transiciones que corresponden a la fracción f . Los términos se definen en la Sección 2.2 (ϵ -greedy, red objetivo, recorte del gradiente y PBRs).

	DQN	DQN recurrente	Transformer
Parámetros entrenables	5 131	34 027	27 019
Tasa de aprendizaje	0,001508	0,002415	0,000215
Descuento γ	0,9634	0,9719	0,9234
ϵ_0 / $\epsilon_{\min}$	0,9627 / 0,0691	0,8883 / 0,0887	0,8651 / 0,0838
Fracciones w / q	0,2779 / 0,6290	0,1237 / 0,5990	0,2428 / 0,5333
Tamaño de lote	128	64	128
Capacidad del <i>buffer</i>	20 000	60 000	30 000
Sincronización C (pasos)	1 000	2 000	500
Recorte del gradiente	3,953	3,475	4,170
PBRs κ	0,0226	0,00263	0
Inicio del aprendizaje f	0,1615 (3 230)	0,2021 (12 128)	0,1217 (3 650)
Episodios	40 000	30 000	30 000

Tabla 4.4: Hiperparámetros de A2C (GAE, coeficiente de entropía y PBRs: Secciones 2.2 y 2.3).

Tasa de aprendizaje inicial (actor / crítico)	0,000648 / 0,004299
Decaimiento de la tasa	coseno, hasta el 23,75 % del valor inicial
Descuento γ / λ de GAE	0,8545 / 0,9135
Coeficiente de entropía β_H	0,00528
Recorte del gradiente (ambas redes)	1,200
PBRs κ	0,1527
Penalización de gasto	$r \leftarrow r - 0,01039 a$
Sesgo inicial del logit de $a = 10$	-1,389
Episodios	125 000

4.5. Ensamblés

Los ensambles no se entrenan: cargan los modelos ya entrenados de DQN, DQN recurrente y Transformer, y cinco de las seis variantes suman el actor de A2C. Para cada miembro k , en un estado con R recursos, la distribución sobre las acciones factibles se obtiene de

sus valores Q con una *softmax* de temperatura τ (Sección 2.5):

$$p_k(a \mid s) = \frac{\exp(Q_k(s, a)/\tau)}{\sum_{a' \leq R} \exp(Q_k(s, a')/\tau)} \quad a \leq R \quad (4.7)$$

Donde $Q_k(s, a)$ es el valor que el miembro k asigna a la acción a ; en A2C, p_k es la salida del actor renormalizada sobre las acciones factibles. De p_k se obtienen la acción preferida del miembro, $a_k = \arg \max_a p_k(a \mid s)$, y su confianza:

$$c_k = 1 - \frac{H(p_k)}{\log |\mathcal{A}_R|} \quad (4.8)$$

Donde $H(p_k)$ es la entropía de Shannon de p_k (Ecuación (2.4)) y $|\mathcal{A}_R|$ el número de acciones factibles; el ensamble ponderado normaliza por $\log 11$. Con $R = 0$ la única acción factible es $a = 0$ y se ejecuta sin votar. El peso efectivo de cada miembro es $\tilde{w}_k = w_k c_k^\rho$, con w_k un peso base y $\rho \geq 0$ un exponente que regula la influencia de la confianza (si $\rho = 0$, la ignora). Algunas reglas usan también $\hat{Q}_k(a)$, los valores Q del miembro estandarizados sobre las acciones factibles (en A2C, sus probabilidades).

Ensamble ponderado. Promedia las distribuciones de DQN, DQN recurrente y Transformer con pesos que crecen con la confianza:

$$p_{\text{ens}}(a \mid s) \propto \eta(a) \sum_{k=1}^3 w_k (0,1 + c_k) p_k(a \mid s) \quad \eta(a) = \begin{cases} 0,3 & a = 0, R > 0 \\ 1 & \text{en otro caso} \end{cases} \quad (4.9)$$

Donde w_k son los pesos base (0,18, 0,90 y 5,0 para DQN, DQN recurrente y Transformer, normalizados a suma 1, de modo que el Transformer tiene el 82 %), la *softmax* usa $\tau = 15$ y $\eta(a)$ reduce la probabilidad de no asignar nada mientras queden recursos. Esa distribución se mezcla con un *prior* centrado en la severidad actual:

$$p'(a \mid s) \propto (1 - w_\pi) p_{\text{ens}}(a \mid s) + w_\pi \pi_{\text{prior}}(a \mid s) \quad \pi_{\text{prior}}(a \mid s) \propto \exp \left[-\frac{(a - S)^2}{2\sigma_S^2} \right] \quad (4.10)$$

Donde S es la severidad de la ciudad actual, recortada a 9 (el ensamble recorta S a 9 también en el estado que reciben sus miembros), $\sigma_S = 3$ el ancho del *prior* y $w_\pi = 0,17$ su peso; las dos distribuciones se normalizan sobre las acciones factibles. La acción ejecutada es la de mayor p' , salvo que $S \geq 6$ y esa acción sea menor que $\lfloor S/2 \rfloor$: entonces una cota de seguridad la reemplaza por $\lfloor S/2 \rfloor$.

Voto suave. Elige la acción de mayor $\sum_k \tilde{w}_k p_k(a | s)$. Es la única variante optimizable que ajusta la temperatura ($\tau = 1,219$); las otras cuatro usan $\tau = 1$.

Voto por acción. Cada miembro vota por a_k con peso $\tilde{w}_k$; los empates se resuelven con la suma de los $\hat{Q}_k$.

Consenso. Al voto ponderado le suma β_a por la confianza de los miembros que prefieren cada acción, le resta β_d por la de los que prefieren otra y agrega $\sum_k c_k \hat{Q}_k(a)$.

Consenso con *prior*. Conserva los términos β_a y β_d del consenso, pero reemplaza el voto y los $\hat{Q}_k$ por la suma $\sum_k \tilde{w}_k p_k$, atenúa la acción nula como η , mezcla el resultado con el *prior* de la Ecuación (4.10), con sus propios w_π y σ_S , y aplica la misma cota de seguridad.

Con compuerta para el Transformer. Calcula la compuerta logística $g = 1/\{1 + \exp[-\kappa_g(c_{\text{trf}} - \tau_g)]\}$, con c_{trf} la confianza del Transformer, umbral $\tau_g = 0,2222$ y pendiente κ_g ; si $g \geq 0,5$ ejecuta la acción del Transformer y, si no, la de un voto suave con todos los miembros. En el resto del texto se lo denomina *compuerta del Transformer*.

Los parámetros del ensamble ponderado se fijaron a mano, observando su desempeño en las 64 secuencias de la referencia, y no provienen de una búsqueda sistemática; la Sección 5.5 los justifica a posteriori con un análisis de sensibilidad y con la asignación óptima. Los de los otros cinco, incluido el peso de A2C, son los del mejor ensayo de Optuna sobre esas mismas secuencias (Tabla 4.5). En los cinco, el puntaje del estudio coincide con la media del ensamble en la referencia. Como las reglas no se entrenan junto con los miembros, las diferencias entre ensambles miden el efecto de la regla sobre los mismos modelos.

Tabla 4.5: Parámetros de los ensambles: temperatura τ , exponente de confianza ρ y pesos base de DQN, DQN recurrente, Transformer y A2C.

Ensamble	τ	ρ	w_{DQN}	w_{rec}	w_{trf}	w_{A2C}	Otros
Ponderado	15	—	0,18	0,90	5,0	—	$w_\pi = 0,17, \sigma_S = 3$
Voto suave	1,219	2,681	0,270	1,760	0,896	0,0070	—
Voto por acción	1	2,197	0,468	2,599	1,803	0,174	—
Consenso	1	0,2754	1,532	1,931	2,504	1,881	$\beta_a = 2,074,$ $\beta_d = 0,1211$
Consenso con prior	1	1,050	2,122	2,723	2,277	0,3366	$\beta_a = 0,2437,$ $\beta_d = 0,1146,$ $w_\pi = 0,1777,$ $\sigma_S = 0,9097$
Compuerta Transformer	1	1,452	0,837	2,561	2,601	0,0674	$\tau_g = 0,2222,$ $\kappa_g = 2,397$

4.6. Optimización de hiperparámetros

Cada paquete optimizable ejecuta un estudio de Optuna con TPE [27, 28] que, en cada ensayo, entrena el agente desde cero y maximiza el $\bar{r}$ medio sobre las 64 secuencias de referencia; los ensayos poco prometedores se interrumpen con **MedianPruner** y el número de episodios forma parte de la búsqueda. El modelo final se reentrena con la mejor configuración y la semilla de su ensayo.

4.7. Escenarios de generalización

Además de la referencia, cada modelo se evalúa en 21 escenarios de generalización (Tabla 4.6): 9 cambian la distribución de severidades, 5 las longitudes, 4 ambas y 3, de control, repiten las secuencias de la referencia, dos con otra partición en bloques. Ocho de los escenarios de severidad incluyen valores de 0, 1 o 9, que están en el espacio de estados pero no en el entrenamiento. Tres escenarios quedan *fuera de rango*: la *extrapolación de severidad* (severidades de 10 a 12), las *secuencias largas* (11 a 20 pasos) y la *extrapolación conjunta* (ambas). Todos se generan con la semilla 42, por lo que los modelos enfrentan las mismas secuencias. La *degradación* en un escenario es $\bar{r}_{\text{ref}} - \bar{r}_{\text{esc}}$ (positiva si se pierde desempeño).

Como las optimizaciones eligen su configuración sobre las mismas 64 secuencias de la referencia, se agregan cinco *réplicas fuera de muestra* (**heldout_s1** a **heldout_s5**). Cada una sortea 8×8 secuencias nuevas: cada longitud y cada severidad inicial, de forma independiente, con las frecuencias observadas en la referencia, que son las mismas con las

que se entrenan los modelos individuales. Las réplicas no intervienen en el entrenamiento, en la optimización ni en las medias de generalización; la diferencia entre la referencia y su media mide cuánto se sobreajustó cada configuración a esas 64 secuencias.

Tabla 4.6: Referencia, 21 escenarios de generalización y 5 réplicas fuera de muestra. Los que tienen **extrap** en el nombre superan las cotas del entorno ($S \leq 9$, $L \leq 10$).

Familia	Identificador	Descripción
referencia	sev_base	Distribución de entrenamiento.
severidad	sev_uniform	$U\{0, \dots, 9\}$.
severidad	sev_gauss_low	$\mathcal{N}(2, 1, 5)$ truncada, brotes leves.
severidad	sev_gauss_mid	$\mathcal{N}(4, 5, 2, 0)$ truncada, brotes medios.
severidad	sev_gauss_high	$\mathcal{N}(7, 1, 5)$ truncada, brotes intensos.
severidad	sev_weibull	Weibull($k = 1, 5$), cola superior pesada.
severidad	sev_beta_lowskew	$9 \cdot \text{Beta}(2, 5)$, sesgo bajo.
severidad	sev_beta_highskew	$9 \cdot \text{Beta}(5, 2)$, sesgo alto.
severidad	sev_bimodal	$0,5\mathcal{N}(2, 1) + 0,5\mathcal{N}(7, 1)$.
severidad	sev_extrapolate_high	$U\{10, 11, 12\}$, fuera de rango.
longitud	len_all_short	Todas las secuencias de longitud 3.
longitud	len_all_long	Todas las secuencias de longitud 10.
longitud	len_geometric	$2 + \text{Geom}(p = 0, 2)$, recortada a $[3, 10]$.
longitud	len_poisson	$\text{Poisson}(\lambda = 5)$, recortada a $[3, 10]$.
longitud	len_extrapolate_long	$U\{11, \dots, 20\}$, fuera de rango.
conjunta	joint_high_long	Gaussiana alta $\times$ todas largas.
conjunta	joint_low_short	Gaussiana baja $\times$ todas cortas.
conjunta	joint_uniform_geom	Uniforme $\times$ geométrica.
conjunta	joint_extrap_both	Severidad y longitud fuera de rango.
estructural	struct_few_long_blocks	4 bloques $\times$ 16 secuencias.
estructural	struct_many_short_blocks	16 bloques $\times$ 4 secuencias.
estructural	struct_more_total	8 bloques $\times$ 8 secuencias, las de la referencia.
réplica	heldout_s1-s5	Sorteo independiente de longitudes y severidades con las frecuencias de la referencia (semilla $42 + k$).

4.8. Protocolo de evaluación

Cada modelo se evalúa en los 22 escenarios y en las 5 réplicas de forma determinista, con la acción factible de mayor valor (Apéndice B); los modelos individuales, con Q-Learning base, y los ensambles forman dos grupos. Para cada modelo y escenario se calculan la media y la desviación estándar de $\bar{r}$ y la degradación; las comparaciones entre dos modelos juntan las secuencias de los 21 escenarios de generalización y se resumen con la d de Cohen y el valor p de Welch (Sección 2.7).

5. Resultados

Los valores a los que se hace referencia, salvo que se especifique otra cosa, corresponden al desempeño normalizado $\bar{r}$ (Ecuación (4.5)) medio de cada escenario; *referencia* es la condición de entrenamiento y *generalización*, la media en los 21 escenarios. Los mapas estadísticos por escenario y las matrices de comparación entre pares pueden encontrarse en el Apéndice C (Figuras C.1–C.5).

5.1. Mejor modelo individual

El Transformer demuestra ser el mejor modelo individual (Tabla 5.1): 0,927 en la referencia y 0,930 en generalización, frente a 0,871–0,899 del resto de los modelos optimizados, con la menor dispersión entre secuencias y la mayor media en 16 de los 22 escenarios (Figura 5.1). Su ventaja es de tamaño medio frente a DQN ($d = 0,50$), al DQN recurrente ($d = 0,57$) y a Q-Learning ($d = 0,79$).

Tabla 5.1: Modelos individuales: media en la referencia, desviación estándar entre secuencias (σ), media en generalización, peor escenario y pérdida respecto de la referencia.

Modelo	Ref.	σ	Gen.	Peor escenario	Pérdida
Transformer	0,927	0,045	0,930	Secuencias largas (0,860)	0,068
DQN	0,894	0,055	0,899	Secuencias largas (0,841)	0,053
A2C	0,887	0,063	0,896	Secuencias largas (0,826)	0,062
DQN recurrente	0,899	0,049	0,889	Extrap. de severidad (0,831)	0,068
Double Q-Learning	0,896	0,048	0,877	Extrap. de severidad (0,785)	0,111
Q-Learning	0,887	0,061	0,871	Extrap. de severidad (0,762)	0,124
Q-Learning base	0,871	0,074	0,851	Extrap. de severidad (0,627)	0,244

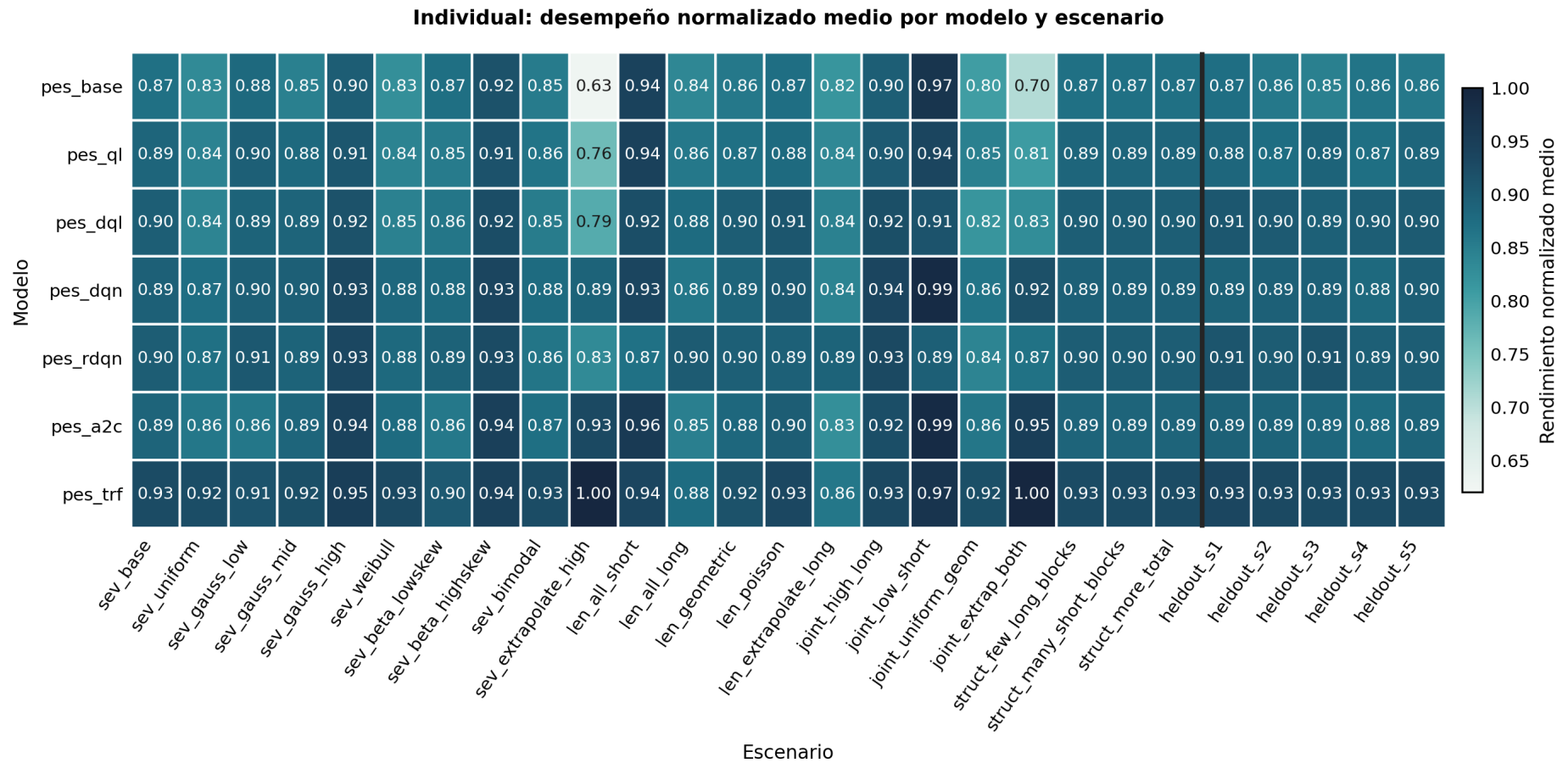


Figura 5.1: Modelos individuales: desempeño medio $\bar{r}$ por modelo y escenario. La primera columna es la referencia. Las cinco columnas a la derecha de la línea vertical son las réplicas fuera de muestra.

En la Figura 5.2, con las secuencias ordenadas de menor a mayor $\bar{r}$, el Transformer queda por encima o al nivel del resto en los tres escenarios de severidad y en el de longitudes Poisson, con la mayor ventaja en los peores casos de la distribución bimodal; con secuencias largas lo supera el DQN recurrente, y con alta severidad y longitud 10 las curvas casi se superponen. Las Figuras D.1–D.7 (Apéndice D) muestran, para cada modelo, el desempeño de cada secuencia de la referencia, su distribución y sus estadísticos por bloque.

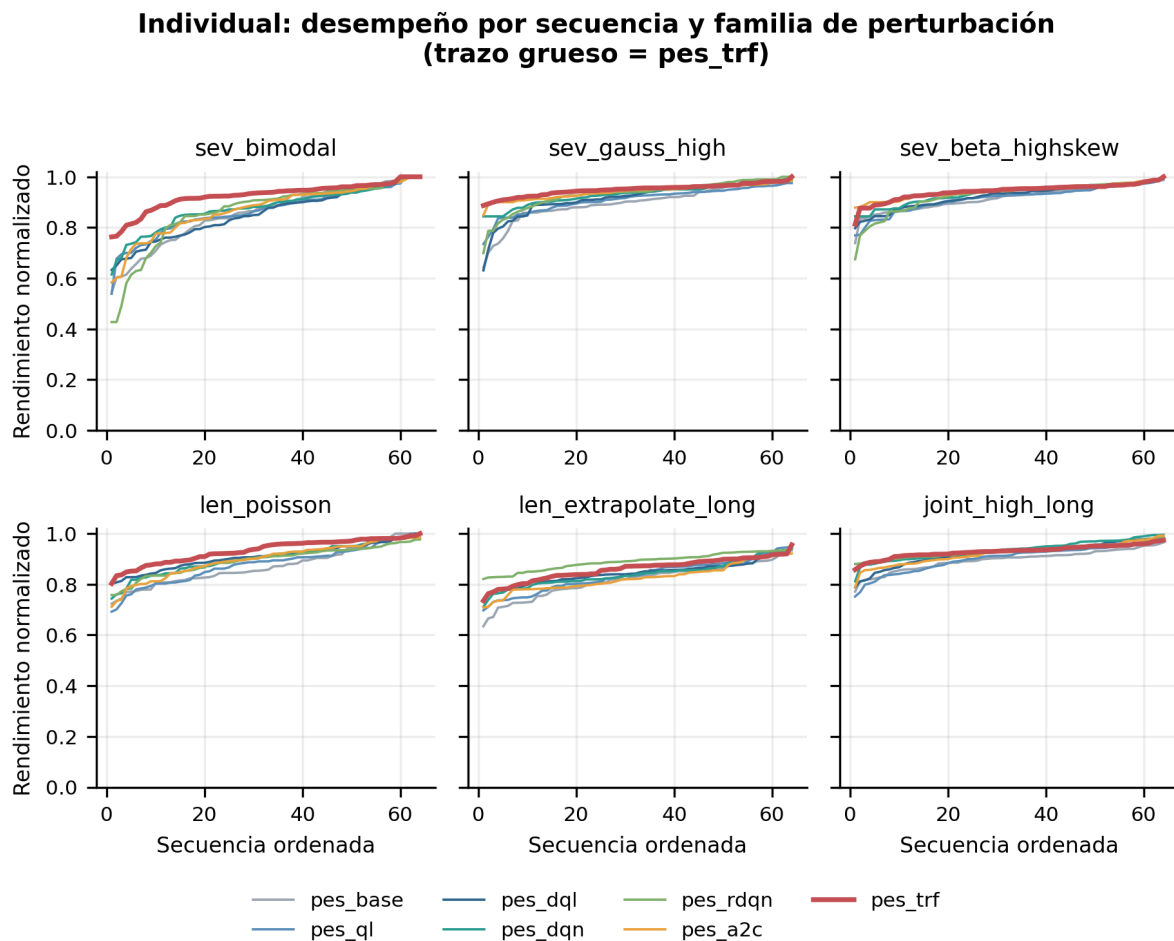


Figura 5.2: Desempeño por secuencia, ordenado de menor a mayor, en seis escenarios de generalización. Fila superior: severidad; fila inferior: longitud y conjunta. El trazo grueso corresponde al Transformer.

5.2. Ensamblas

La Tabla 5.2 resume los seis ensambles y la Tabla 5.3 compara cada uno con el Transformer individual; como los dos grupos se evaluaron por separado, d y p se calculan con la media, la desviación estándar y el número de secuencias de cada escenario.

Sólo el ensamble ponderado supera al Transformer: 0,939 frente a 0,930 en generalización,

mayor media en 18 de los 21 escenarios y un peor escenario 0,040 más alto. La diferencia es significativa pero de tamaño despreciable ($d = 0,19$, apenas por debajo de un efecto pequeño). En la referencia reduce en un 45 % la distancia al óptimo que deja Q-Learning, $(\bar{r} - \bar{r}_{ql}) / (1 - \bar{r}_{ql})$, frente al 36 % del Transformer. Frente a los demás ensambles, su ventaja es de tamaño medio sobre el consenso ($d = 0,58$) y despreciable sobre la compuerta ($d = 0,17$); ambas son significativas ($p < 10^{-5}$).

Tabla 5.2: Ensamblas: media en la referencia, media en generalización, peor escenario y pérdida respecto de la referencia.

Ensamble	Ref.	Gen.	Peor escenario	Pérdida
Ensamble ponderado	0,937	0,939	Secuencias largas (0,900)	0,037
Compuerta del Transformer	0,928	0,931	Secuencias largas (0,861)	0,067
Consenso con prior	0,918	0,919	Secuencias largas (0,870)	0,047
Consenso	0,889	0,905	Secuencias largas (0,826)	0,063
Voto suave	0,914	0,902	Extrap. de severidad (0,860)	0,054
Voto por acción	0,914	0,901	Extrap. de severidad (0,859)	0,055

Tabla 5.3: Cada ensamble frente al Transformer individual en generalización; los valores positivos indican ventaja del ensamble.

Ensamble	Diferencia	d	Efecto	p	Mayor media en
Ensamble ponderado	+0,010	0,19	despreciable	$< 10^{-5}$	18 de 21
Compuerta del Transformer	+0,001	0,02	despreciable	0,68	15 de 21
Consenso con prior	-0,010	-0,18	despreciable	$< 10^{-5}$	5 de 21
Consenso	-0,025	-0,39	pequeño	$< 10^{-22}$	3 de 21
Voto suave	-0,027	-0,40	pequeño	$< 10^{-24}$	2 de 21
Voto por acción	-0,028	-0,42	pequeño	$< 10^{-26}$	2 de 21

La compuerta del Transformer obtiene un desempeño equivalente al del Transformer ($d = 0,02$; $p = 0,68$): supera su media en 15 escenarios, pero nunca por más de 0,006. Los otros cuatro ensambles quedan por debajo del Transformer individual. El voto suave y el voto por acción, que sólo promedian o votan ponderando por la confianza, tienen las medias más bajas (efecto pequeño, $d \approx -0,4$). Su peor escenario es la extrapolación de

severidad: el 41 % y el 42 % de sus secuencias quedan por debajo de 0,8 (Figura 5.4) y su media queda por debajo de la de tres de sus cuatro miembros: el Transformer (0,996), A2C (0,929) y DQN (0,890).

El mejor ensamble es el ponderado: tiene la mayor media en la referencia (0,937) y en generalización (0,939), el peor escenario más alto (0,900, secuencias largas) y la menor pérdida respecto de la referencia (0,037) (Tabla 5.2). Es además el ensamble con mayor media en 19 de los 22 escenarios; sólo lo superan el consenso, en `len_all_short` y `joint_low_short`, y la compuerta del Transformer, en `len_poisson` (Figura 5.3).

5.2.1. Efecto de las reglas fijas sobre la decisión

Tres ensambles agregan reglas fijas al voto: el ensamble ponderado y el consenso con *prior* lo mezclan con un *prior* de severidad y aplican una cota de seguridad, y la compuerta del Transformer decide cuándo ejecutar la acción del Transformer; el voto suave, el voto por acción y el consenso no tienen reglas fijas. Para saber si la decisión de esos tres ensambles proviene del voto o de las reglas, se repitió su evaluación registrando, en cada decisión, la acción antes y después de cada regla y la que prefiere el Transformer (Apéndice B). La repetición reproduce las medias de la evaluación principal con diferencias menores que el 0,01 % de la escala de $\bar{r}$ (Tabla 5.4).

En el consenso con *prior* decide el voto: el *prior* cambia la acción votada en menos del 5 % de las decisiones, tanto en la referencia como en generalización, y el ensamble queda por debajo del Transformer. En la compuerta decide el Transformer: su acción se ejecuta en el 97 % y el 95 % de las decisiones, lo que explica la equivalencia de sus desempeños. En el ensamble ponderado deciden el voto y el *prior*: con $\tau = 15$ las distribuciones de los miembros quedan casi planas (en la referencia, la acción más probable de cada uno tiene en promedio probabilidad 0,19 y supera a la segunda por 0,02–0,03), y el *prior* cambia la acción votada en el 41 % y el 32 % de las decisiones. La cota casi nunca actúa. Por lo tanto, en los dos ensambles que igualan o superan al Transformer la decisión no proviene sólo del voto ponderado por confianza, sino de la regla.

Tabla 5.4: Porcentaje de las decisiones con recursos disponibles en que ocurre cada evento.

Ensamble	Evento	Ref.	Gen.
Compuerta del Transformer	Sigue al Transformer ($g \geq 0,5$)	96,9 %	94,9 %
	Acción final distinta de la del Transformer	0,8 %	2,7 %
Consenso con prior	El <i>prior</i> cambia la acción votada	4,5 %	4,6 %
	La cota cambia la acción	0,3 %	1,5 %
	Acción final distinta de la del Transformer	62,4 %	58,4 %
Ensamble ponderado	El <i>prior</i> cambia la acción votada	40,9 %	32,1 %
	La cota cambia la acción	0,0 %	0,3 %
	Acción final distinta de la del Transformer	61,2 %	47,4 %

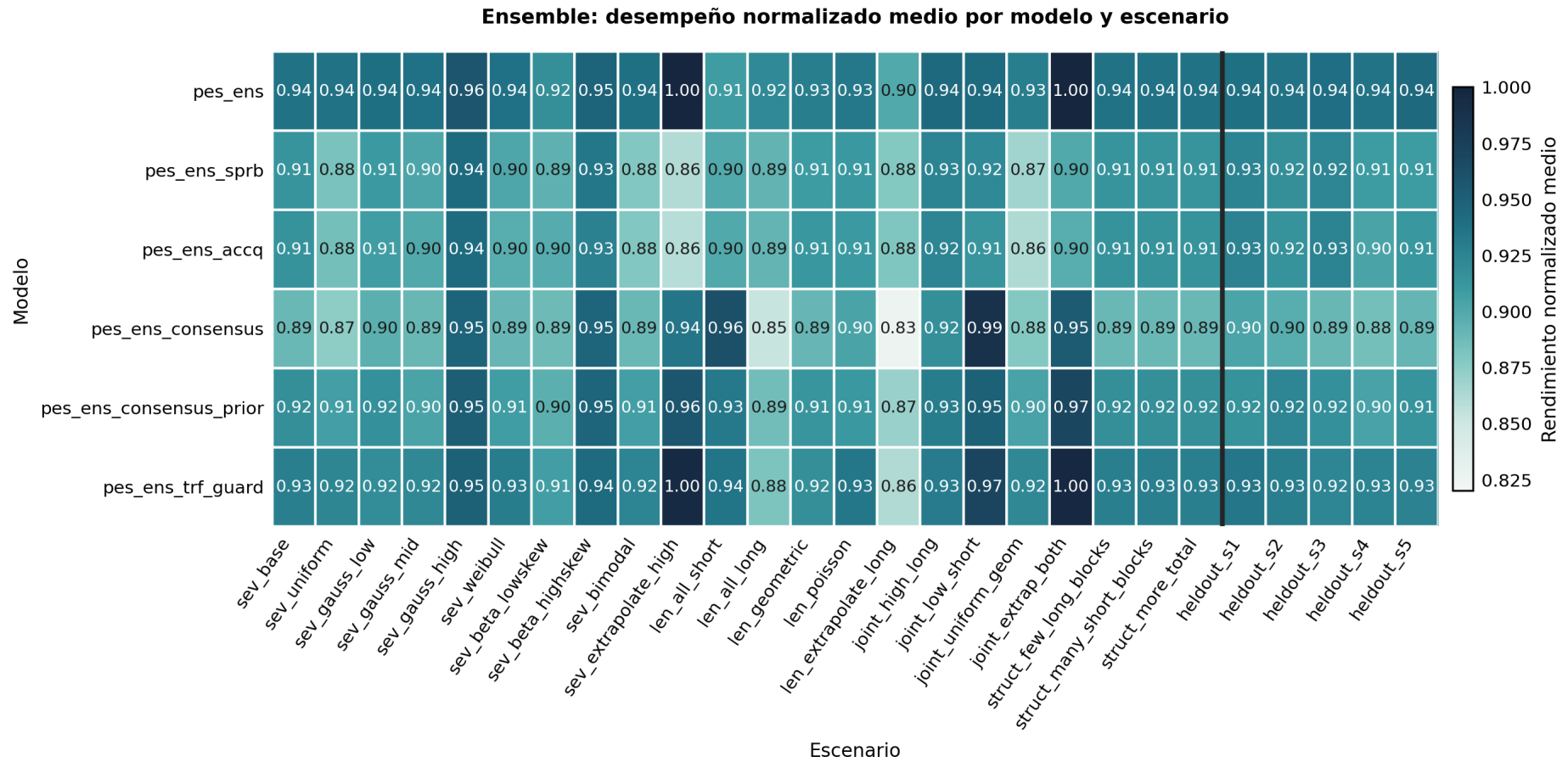


Figura 5.3: Ensembles: desempeño medio $\bar{r}$ por ensemble y escenario. La primera columna es la referencia. Las cinco columnas a la derecha de la línea vertical son las réplicas fuera de muestra.

5.3. Transformer frente al mejor ensamble

El ensamble ponderado supera al Transformer en todas las medidas de la Tabla 5.5: +0,010 en la media de generalización, con un efecto despreciable ($d = 0,19$; Tabla 5.3), +0,040 en el peor escenario y una pérdida máxima de poco más de la mitad. El Transformer tiene mayor media sólo en `len_all_short` (0,936 frente a 0,908), en `joint_low_short` (0,969 frente a 0,942) y, por menos de 0,001, en `len_poisson`. En la extrapolación de severidad y en la conjunta, el ensamble ponderado alcanza el óptimo en todas las secuencias ($\sigma = 0$; Figura 5.4), pero todas sus decisiones coinciden con las del Transformer que lo integra: la diferencia con el Transformer individual proviene de que el ensamble recorta la severidad a 9 antes de pasársela a sus miembros. La Figura 5.5 muestra su desempeño por secuencia en la referencia, donde tiene la menor dispersión de todos los sistemas ($\sigma = 0,035$).

Tabla 5.5: Transformer individual frente al ensamble ponderado, el mejor ensamble.

Medida	Transformer	Ensamble ponderado
Media en la referencia (σ)	0,927 (0,045)	0,937 (0,035)
Media en generalización	0,930	0,939
Peor escenario (secuencias largas)	0,860	0,900
Mayor pérdida	0,068	0,037
Extrapolación de severidad	0,996	1,000
Extrapolación conjunta	0,997	1,000
Escenarios con mayor media (de 21)	3	18

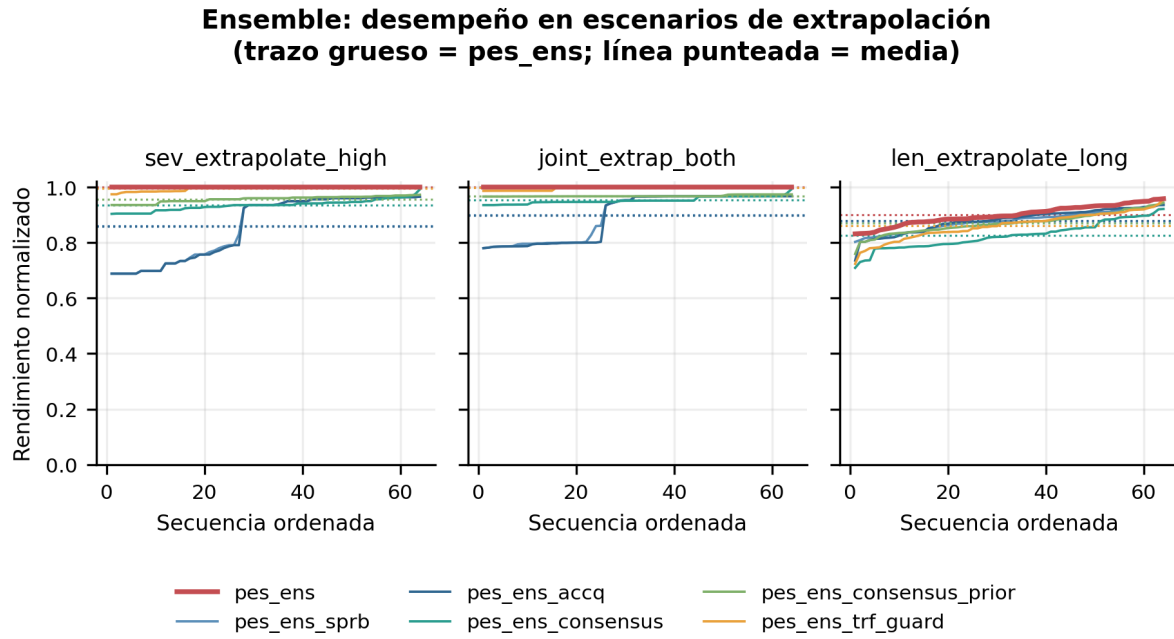


Figura 5.4: Desempeño por secuencia, ordenado de menor a mayor, de los seis ensambles en los escenarios fuera de rango; trazo grueso: ensemble ponderado; líneas punteadas: media de cada ensemble.

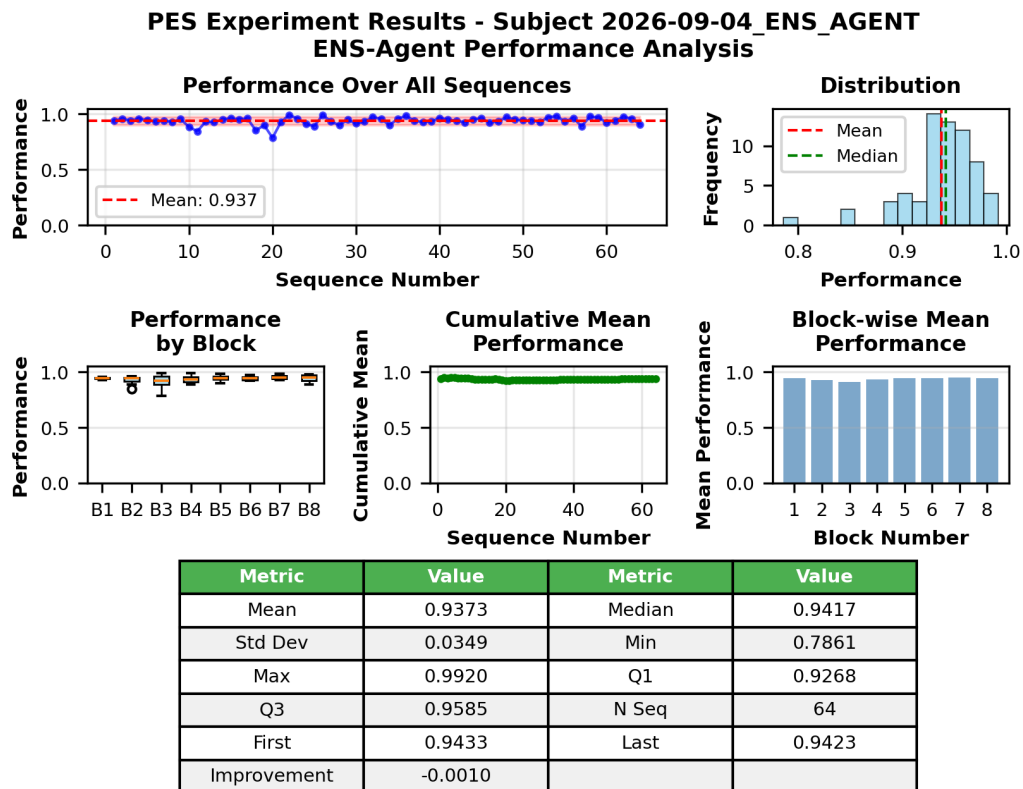


Figura 5.5: Resultados por secuencia del ensemble ponderado (pes_ens) en la referencia.

5.4. Réplicas fuera de muestra

En las cinco réplicas fuera de muestra, 320 secuencias nuevas de la distribución de la referencia, ningún modelo pierde desempeño de forma apreciable (Tabla 5.6). La caída respecto de la referencia va de $-0,004$ a $+0,011$, siempre con efecto despreciable ($|d| \leq 0,15$) y sin significación ($p \geq 0,29$). En los once modelos optimizados con Optuna queda entre $-0,004$ y $+0,005$; la mayor corresponde a Q-Learning base, que no se optimizó, de modo que diferencias de ese tamaño son compatibles con la variación entre muestras. El ordenamiento se conserva (correlación de Spearman entre las medias, $r_s = 0,97$): sólo cambian de posición modelos cuyas medias difieren en menos de $0,005$ en ambas condiciones. En las mismas 320 secuencias, el ensamble ponderado supera al Transformer por $0,010$ (prueba t pareada, $p < 10^{-7}$) y tiene mayor media en las cinco réplicas. Elegir la configuración sobre las 64 secuencias de la referencia no infló, por lo tanto, el desempeño medido en esa condición.

Tabla 5.6: Media en la referencia y en las cinco réplicas fuera de muestra (entre paréntesis, desviación estándar de las medias de las réplicas); caída: referencia menos réplicas; d : réplicas frente a referencia.

Modelo	Ref.	Fuera de muestra	Caída	d
Ensamble ponderado	0,937	0,939 (0,003)	$-0,002$	$+0,04$
Compuerta del Transformer	0,928	0,928 (0,004)	$0,000$	$-0,01$
Transformer	0,927	0,929 (0,003)	$-0,001$	$+0,03$
Consenso con prior	0,918	0,915 (0,007)	$+0,003$	$-0,07$
Voto por acción	0,914	0,916 (0,009)	$-0,002$	$+0,04$
Voto suave	0,914	0,917 (0,008)	$-0,003$	$+0,06$
DQN recurrente	0,899	0,903 (0,008)	$-0,004$	$+0,08$
Double Q-Learning	0,896	0,900 (0,006)	$-0,004$	$+0,07$
DQN	0,894	0,891 (0,006)	$+0,002$	$-0,04$
Consenso	0,889	0,893 (0,007)	$-0,004$	$+0,06$
A2C	0,887	0,887 (0,006)	$0,000$	$-0,01$
Q-Learning	0,887	0,881 (0,008)	$+0,005$	$-0,08$
Q-Learning base	0,871	0,860 (0,010)	$+0,011$	$-0,15$

5.5. Justificación a posteriori del ensamble ponderado

Como los parámetros del ensamble ponderado se fijaron a mano (Sección 4.5), se analizaron después de la evaluación, sin modificarlos, de dos maneras: variando cada uno por separado y contrastando cada regla fija con la asignación óptima. Ambos análisis usan la referencia y las cinco réplicas fuera de muestra (Apéndice B).

Sensibilidad. La Tabla 5.7 varía un parámetro por vez y deja los demás en sus valores fijos. La temperatura, el peso y el ancho del *prior* y el peso base del Transformer tienen su máximo en el valor elegido, tanto en la referencia como en las réplicas, de modo que la elección no depende de las 64 secuencias sobre las que se hizo; pero son máximos angostos: un paso de la grilla cuesta entre 0,002 y 0,008. La atenuación η , el término 0,1 y la cota de seguridad están en una meseta: cambian la media en a lo sumo 0,001. En las réplicas, sin *prior* ($w_\pi = 0$) el ensamble supera al Transformer por 0,004; con el *prior* y $\tau = 1$, por 0,001, y con ambos, por 0,010. La ventaja proviene, por lo tanto, de combinar la temperatura alta con el *prior*.

Tabla 5.7: Sensibilidad del ensamble ponderado a cada parámetro, con los demás en sus valores fijos. La última columna indica los valores con los que supera al Transformer en las réplicas fuera de muestra por más de dos errores estándar pareados.

Parámetro	Fijo	Probados	Máximo	Supera al Transformer
Temperatura τ	15	1 a 50 (8)	15	2 a 30
Peso del <i>prior</i> w_π	0,17	0 a 0,5 (7)	0,17	0 a 0,25
Ancho del <i>prior</i> σ_S	3	1 a 5 (5)	3	3 a 5
Peso base del Transformer	5	1 a 10 (4)	5	2,5 a 10
Atenuación η	0,3	0,1 a 1 (4)	meseta	todos
Término $0,1 + c_k$	0,1	0 a 1 (4)	meseta	todos
Cota de seguridad	activa	sí / no	sin efecto	ambos

Contraste con el óptimo. La asignación óptima de cada ciudad se reconstruyó con la programación dinámica que calcula S_{mejor} (Ecuación (4.6)), guardando en cada ciudad la acción que alcanza el mínimo, en las 384 secuencias de la referencia y de las réplicas; en el 6 % de las ciudades, todas con $S \leq 5$, hay más de una acción óptima, y los resultados no cambian al considerarlas todas. Mientras queda presupuesto, el óptimo nunca asigna 0 y

asigna en promedio entre 0,7 y 1,9 unidades más que la severidad de la ciudad; cuando el presupuesto se agota, asigna 0. El *prior* acierta el ancho pero no el centro: una gaussiana centrada en S ajustada a esas asignaciones tiene $\sigma_S = 2,7$, cercano al 3 elegido, pero si también se ajusta el centro, este queda en $S + 2,2$, con $\sigma_S = 1,4$, y el ajuste mejora mucho; el ancho del *prior* compensa en parte su centro desplazado. La cota nunca contradice al óptimo, que asigna al menos $\lfloor S/2 \rfloor$ siempre que puede, pero el ensamble casi nunca queda por debajo de ella: cambia 2 de 1 726 decisiones. La atenuación η va en el sentido del óptimo, con un efecto despreciable. El *prior* es la única regla con efecto apreciable: cambia el 37 % de las decisiones con recursos y mejora $\bar{r}$ en 0,012 por secuencia.

Una regla fija como política. Si el óptimo asigna algo más que la severidad, cabe preguntar cuánto rinde hacerlo sin ningún modelo. La Tabla 5.8 evalúa la regla $a = \min(S + k, R)$, que asigna la severidad de la ciudad más k unidades mientras alcancen los recursos, en las mismas secuencias que los modelos. Con $k = 2$ iguala al ensamble ponderado y supera al Transformer en 18 de los 21 escenarios de generalización y en las cinco réplicas. Pierde con secuencias cortas, en las que le sobra presupuesto (0,845 frente a 0,936 del Transformer en `len_all_short`), y con secuencias largas le conviene un k menor. El valor $k = 2$ se leyó de la asignación óptima, que usa información completa; es también el mejor en la referencia, la condición sobre la que se ajustaron los modelos. La versión que no requiere conocer el óptimo, asignar la severidad ($k = 0$), queda muy por debajo de todos los modelos entrenados.

Tabla 5.8: Desempeño medio de la regla fija $a = \min(S + k, R)$, sin modelo, frente al Transformer y al ensamble ponderado en las mismas secuencias.

Política	Ref.	Gen.	Réplicas
Regla, $k = 0$ (asigna S)	0,782	0,805	0,787
Regla, $k = 1$	0,900	0,907	0,906
Regla, $k = 2$	0,940	0,937	0,943
Regla, $k = 3$	0,939	0,940	0,941
Transformer	0,927	0,930	0,929
Ensamble ponderado	0,937	0,939	0,939

6. Discusión

Los resultados respaldan la hipótesis H1 y no respaldan la H2: sólo una de las seis reglas, el ensamble ponderado, supera al Transformer, con una diferencia de tamaño despreciable (Sección 6.2). Una regla fija calibrada con la asignación óptima alcanza el mismo nivel, lo que acota el alcance de ambas conclusiones (Sección 6.3).

6.1. Interpretación de la ventaja del Transformer

La ventaja del Transformer no puede atribuirse a la ventana de estados: el DQN recurrente recibe la misma ventana de seis estados y en generalización queda un 4 % por debajo del Transformer e incluso un 1 % por debajo del DQN, que sólo observa el estado actual. Tampoco se debe al tamaño, porque el Transformer tiene un 21 % menos de parámetros que el DQN recurrente. Descartados esos dos factores, la mejora se atribuye a la atención causal, que pondera cada estado de la ventana según su relación con el estado actual en lugar de resumirla en un único estado oculto, como hace la LSTM. La ventaja se conserva en generalización: el Transformer obtiene la mayor media en el 73 % de los escenarios (16 de 22; Figura 5.1) y su media allí supera en un 0,3 % a la de la referencia. Su mayor pérdida ocurre con secuencias más largas que las de entrenamiento, donde cae un 7 % y queda un 3 % por debajo del DQN recurrente. Como no se evaluó una variante sin atención, su efecto no puede separarse del de las conexiones residuales con normalización de capa ni del de los hiperparámetros.

6.2. Efecto de la regla de combinación

Los seis ensambles combinan los mismos modelos y ponderan a cada miembro por su confianza; sólo cambia la regla (Tabla 5.4):

- En el **ensamble ponderado** la decisión no la determina el Transformer, pese a que tiene el 82 % del peso base: $\tau = 15$ aplanan las distribuciones de los miembros y el *prior* cambia la acción votada en el 41 % de las decisiones de la referencia y el 32 % de las de generalización. Su ventaja se asocia a la combinación de una preferencia débil de los modelos con una regla fija que asigna aproximadamente

según la severidad.

- La **compuerta del Transformer** lo sigue en el 97 % y el 95 % de las decisiones, y por eso alcanza un desempeño equivalente.
- En el **voto suave** y el **voto por acción** los pesos base favorecen al DQN recurrente sobre el Transformer; en la extrapolación de severidad, la ponderación por confianza no produjo que predominara el miembro con mayor desempeño individual.
- El **consenso con *prior*** mejora la referencia y el peor escenario del consenso, aunque su *prior* cambia menos del 5 % de las decisiones; la mejora no puede atribuirse al *prior*.

La ventaja del ensamble ponderado proviene del *prior* y de la temperatura alta más que de la ponderación por confianza, y requiere ambos: en las réplicas fuera de muestra, la temperatura alta sin *prior* deja una ventaja de 0,004 sobre el Transformer, el *prior* con $\tau = 1$, de 0,001, y los dos juntos, de 0,010 (Sección 5.5). Sus parámetros están en máximos angostos, pero esos máximos coinciden en la referencia y en las réplicas, por lo que la ventaja se mantiene fuera de muestra (Sección 5.4). Los datos muestran también que la ponderación por confianza, por sí sola, no basta para superar al mejor miembro: los dos ensambles que sólo promedian o votan con ella quedan por debajo del Transformer individual.

6.3. Alcance de los resultados frente a una regla fija

Que una regla fija de una línea, asignar la severidad más dos unidades, iguale al mejor ensamble y supere al Transformer (Tabla 5.8) obliga a precisar qué muestran los resultados.

No invalida la comparación entre modelos. H1 y H2 comparan arquitecturas y reglas de combinación entre sí, en las mismas condiciones, y esas comparaciones no cambian. Tampoco significa que los agentes no hayan aprendido: la regla necesita conocer de antemano el desplazamiento $k = 2$, que se obtuvo de la asignación óptima con información completa; sin ese dato, asignar la severidad rinde 0,78, mientras que los agentes llegaron a entre 0,85 y 0,93 sólo a partir de la recompensa, sin conocer la regla ni el óptimo. Además, la regla no se adapta: el k que conviene cambia con la longitud de las secuencias, y con secuencias cortas el Transformer la supera ampliamente.

Lo que sí muestra es que la política óptima de mPES tiene una estructura simple, asignar algo más que la severidad mientras dure el presupuesto, y que una heurística calibrada con ella alcanza el nivel de los mejores modelos. En este entorno, por lo tanto, la ventaja práctica de los agentes aprendidos frente a esa heurística es nula: el problema resultó más simple de lo que su formulación sugiere, y la ventaja del ensamble ponderado se explica porque su *prior* lo acerca a esa regla. Es un resultado análogo al de Kong *et al.* [46], en el que el RL redescubre los algoritmos clásicos del problema. Evaluar la utilidad de estos modelos para decisiones reales requiere entornos en los que ninguna regla simple sea casi óptima.

7. Conclusiones

7.1. Respuesta a las preguntas de investigación

Mejor modelo individual. Los resultados respaldan la hipótesis H1. El Transformer causal obtuvo el mayor desempeño medio entre los modelos individuales, en la referencia y en los escenarios de generalización; en estos últimos superó en un 3 % a DQN y en un 5 % al DQN recurrente, diferencias de tamaño medio. Mantuvo el desempeño con severidades fuera de rango, pero no fue el más estable en su peor escenario: con secuencias largas perdió un 7 % respecto de su referencia, frente al 6 % de DQN.

Ensamblés frente al mejor modelo individual. Los resultados no respaldan la hipótesis H2. Sólo el ensamble ponderado superó al Transformer, en un 1 % en generalización, una diferencia significativa pero de tamaño despreciable. La compuerta del Transformer alcanzó un desempeño equivalente y los otros cuatro ensambles quedaron por debajo. La ventaja del ensamble ponderado no se explica por la ponderación por confianza: su acción final difiere de la del Transformer en el 61 % de las decisiones de la referencia, en buena medida por la temperatura alta y el *prior* de severidad. El análisis a posteriori muestra que la ventaja requiere ambos, que el *prior* está centrado por debajo de la asignación óptima y que la cota de seguridad no tiene efecto.

Una regla fija como referencia. Una regla sin modelo que asigna la severidad más dos unidades, calibrada con la asignación óptima, obtiene 0,937 en generalización y 0,943 en las réplicas fuera de muestra: iguala al ensamble ponderado y supera al Transformer. En mPES, por lo tanto, los modelos aprendidos no aportan una ventaja práctica frente a una heurística bien calibrada. La comparación entre arquitecturas y reglas de combinación sigue siendo válida, pero se refiere a un entorno cuya política óptima tiene una estructura simple.

Ponderación por confianza. El diseño no permite determinar si ponderar por la confianza mejora a una votación simple, porque todos los ensambles aplican esa ponderación. Los que sólo promedian o votan con esos pesos obtuvieron un desempeño inferior al del Transformer individual.

7.2. Limitaciones del estudio

- Los escenarios de generalización usan una única semilla (42), lo que impide estimar la variabilidad de sus medias y construir intervalos de confianza para el ordenamiento.
- Cada modelo se entrenó una sola vez, con la semilla de su mejor ensayo; como el RL profundo es sensible a la semilla [50], otro entrenamiento podría cambiar las diferencias pequeñas entre modelos.
- Los hiperparámetros se ajustaron sobre la referencia; las réplicas fuera de muestra (Sección 5.4) no muestran sobreajuste, pero usan las frecuencias de esa referencia.
- Las arquitecturas del DQN recurrente y del Transformer se eligieron *ad hoc*; otras configuraciones podrían modificar el ordenamiento.
- Los parámetros del ensamble ponderado se fijaron a mano sobre la referencia y sólo se justificaron a posteriori; están en máximos angostos de la sensibilidad.
- El entorno admite una regla fija casi óptima, por lo que los resultados no indican cuánto aportan estos modelos en problemas cuya política óptima no tenga una estructura simple.
- La confianza se calcula sobre valores Q transformados y no sobre probabilidades calibradas, y no se verificó su relación con el acierto de las decisiones.

7.3. Trabajo futuro

- Repetir la evaluación con semillas independientes para obtener intervalos de confianza del ordenamiento de la Tabla 5.1.
- Relacionar la confianza con el acierto de cada decisión mediante curvas de calibración.
- Aprender pesos $w_k(s)$ dependientes del estado.
- Incluir como línea de base heurísticas calibradas con el óptimo y evaluar los modelos en variantes del entorno en las que ninguna regla simple sea casi óptima.

7.4. Conclusión general

En el entorno controlado de mPES, el Transformer causal es el modelo individual con mejor desempeño, en las condiciones de entrenamiento y fuera de ellas, aunque no es el que menos se degrada en su peor escenario. La combinación de modelos lo superó en un solo caso y por un margen despreciable, mediante una regla que suaviza las preferencias de los miembros y las mezcla con un *prior* de severidad. Una regla fija calibrada con la asignación óptima alcanza ese mismo nivel, de modo que en este entorno la ventaja de los modelos aprendidos frente a una heurística bien calibrada es nula. En consecuencia, los resultados respaldan H1 y no respaldan H2, dentro de un entorno cuya política óptima es simple. Estas conclusiones se limitan al entorno y a los modelos evaluados.

Referencias Bibliográficas

- [1] BCI-NE Lab, University of Essex. PES: Pandemic Experiment Scenario. <https://github.com/BCI-NE/PES>, 2023. Accedido: septiembre 2026.
- [2] Ellen Kuhl. *Computational Epidemiology: Data-Driven Modeling of COVID-19*. Springer, 2021.
- [3] Andrea Parisi, Samuel P. C. Brand, Joe Hilton, Rabia Aziza, Matt J. Keeling, and D. James Nokes. Spatially resolved simulations of the spread of COVID-19 in three European countries. *PLOS Computational Biology*, 17(7):e1009090, 2021. doi: 10.1371/journal.pcbi.1009090.
- [4] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, MA, 2nd edition, 2018.
- [5] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2): 99–134, 1998. doi: 10.1016/S0004-3702(98)00023-X.
- [6] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8 (3–4):279–292, 1992.
- [7] Hado van Hasselt. Double Q-learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 23, pages 2613–2621, 2010.
- [8] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [9] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992.
- [10] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, pages 2094–2100, 2016.
- [11] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [12] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 9, pages 249–256, 2010.
- [13] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.
- [14] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning (ICML)*, volume 28 of *Proceedings of Machine Learning Research*, pages 1310–1318, 2013.
- [15] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward

- transformations: Theory and application to reward shaping. In *International Conference on Machine Learning (ICML)*, pages 278–287, 1999.
- [16] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- [17] Matthew Hausknecht and Peter Stone. Deep recurrent Q-learning for partially observable MDPs. In *AAAI Fall Symposium Series*, 2015.
- [18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [19] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.
- [20] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, volume 48 of *Proceedings of Machine Learning Research*, pages 1928–1937, 2016.
- [21] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.
- [22] Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.
- [23] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [24] Marco A. Wiering and Hado van Hasselt. Ensemble algorithms in reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 38(4):930–936, 2008. doi: 10.1109/TSMCB.2008.920231.
- [25] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991. doi: 10.1162/neco.1991.3.1.79.
- [26] Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. Safe reinforcement learning via shielding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.11797.
- [27] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 24, pages 2546–2554, 2011.
- [28] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.

- [29] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010.
- [30] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, NJ, 2nd edition, 1988.
- [31] Bernard L. Welch. The generalization of Student’s problem when several different population variances are involved. *Biometrika*, 34(1/2):28–35, 1947.
- [32] Student. The probable error of a mean. *Biometrika*, 6(1):1–25, 1908. doi: 10.2307/2331554.
- [33] Charles Spearman. The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1):72–101, 1904. doi: 10.2307/1412159.
- [34] Akashdeep Nijjar, Ittipat Promnorakid, Riccardo Poli, Caterina Cinel, Thomas Reed, Christopher Baker, Stephen Hinton, and Stephen Fairclough. Enhancing decision-making in human-machine teams. In *2025 IEEE International Conference on Metrology for eXtended Reality, Artificial Intelligence and Neural Engineering (MetroXRaine)*, 2025. doi: 10.1109/metroxraine66377.2025.11340006.
- [35] Tasos Papastylianou, Rodrigo Ramele, Luca Citi, Caterina Cinel, and Riccardo Poli. PES – Pandemic Emergency Scenario. OpenNeuro, conjunto de datos ds004477, versión 1.0.2, 2023. URL <https://doi.org/10.18112/openneuro.ds004477.v1.0.2>.
- [36] Mykel J. Kochenderfer. *Decision Making Under Uncertainty: Theory and Application*. The MIT Press, Cambridge, MA, 2015. doi: 10.7551/mitpress/10187.001.0001.
- [37] Warren B. Powell. *Reinforcement Learning and Stochastic Optimization: A Unified Framework for Sequential Decisions*. Wiley, Hoboken, NJ, 2022. doi: 10.1002/9781119815068.
- [38] Pieter J. K. Libin, Arno Moonens, Timothy Verstraeten, Fabian Perez-Sanjines, Niel Hens, Philippe Lemey, and Ann Nowé. Deep reinforcement learning for large-scale epidemic control. In *Machine Learning and Knowledge Discovery in Databases. Applied Data Science and Demo Track (ECML PKDD 2020)*, volume 12461 of *Lecture Notes in Computer Science*, pages 155–170. Springer, 2021. doi: 10.1007/978-3-030-67670-4_10.
- [39] Abu Quwsar Ohi, M. F. Mridha, Muhammad Mostafa Monowar, and Md. Abdul Hamid. Exploring optimal control of epidemic spread using reinforcement learning. *Scientific Reports*, 10:22106, 2020. doi: 10.1038/s41598-020-79147-8.
- [40] Hamsa Bastani, Kimon Drakopoulos, Vishal Gupta, Ioannis Vlachogiannis, Christos Hadjichristodoulou, Pagona Lagiou, Gkikas Magiorkinis, Dimitrios Paraskevis, and Sotirios Tsiodras. Efficient and targeted COVID-19 border testing via reinforcement learning. *Nature*, 599(7883):108–113, 2021. doi: 10.1038/s41586-021-04014-z.
- [41] Bryan P. Bednarski, Akash Deep Singh, and William M. Jones. On collaborative reinforcement learning to optimize the redistribution of critical medical supplies throughout the COVID-19 pandemic. *Journal of the American Medical Informatics Association*, 28(4):874–878, 2021. doi: 10.1093/jamia/ocaa324.

- [42] Matthieu Komorowski, Leo A. Celi, Omar Badawi, Anthony C. Gordon, and A. Aldo Faisal. The Artificial Intelligence Clinician learns optimal treatment strategies for sepsis in intensive care. *Nature Medicine*, 24(11):1716–1720, 2018. doi: 10.1038/s41591-018-0213-5.
- [43] Chao Yu, Jiming Liu, Shamim Nemati, and Guosheng Yin. Reinforcement learning in healthcare: A survey. *ACM Computing Surveys*, 55(1):5:1–5:36, 2021. doi: 10.1145/3477600.
- [44] Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. Resource management with deep reinforcement learning. In *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets)*, pages 50–56, 2016. doi: 10.1145/3005745.3005750.
- [45] Joren Gijsbrechts, Robert N. Boute, Jan A. Van Mieghem, and Dennis J. Zhang. Can deep reinforcement learning improve inventory management? Performance on lost sales, dual-sourcing, and multi-echelon problems. *Manufacturing & Service Operations Management*, 24(3):1349–1368, 2022. doi: 10.1287/msom.2021.1064.
- [46] Weiwei Kong, Christopher Liaw, Aranyak Mehta, and D. Sivakumar. A new dog learns old tricks: RL finds classic optimization algorithms. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://openreview.net/forum?id=rkluJ2R9KQ>.
- [47] Gabriel Dulac-Arnold, Nir Levine, Daniel J. Mankowitz, Jerry Li, Cosmin Paduraru, Sven Gowal, and Todd Hester. Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. *Machine Learning*, 110(9):2419–2468, 2021. doi: 10.1007/s10994-021-05961-4.
- [48] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision Transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021.
- [49] Emilio Parisotto, H. Francis Song, Jack W. Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant M. Jayakumar, Max Jaderberg, Raphaël Lopez Kaufman, Aidan Clark, Seb Noury, Matthew M. Botvinick, Nicolas Heess, and Raia Hadsell. Stabilizing transformers for reinforcement learning. In *International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 7487–7498, 2020.
- [50] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.11694.
- [51] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 2048–2056, 2020.
- [52] Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 76:201–264, 2023.

-
- [53] Maximiliano Vega. mPES: Multiple Pandemic Experiment Suite. <https://github.com/Maximiliano0/mPES>, 2026. Repositorio oficial. Accedido: septiembre 2026.
- [54] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025. URL <https://arxiv.org/abs/2407.17032>. arXiv:2407.17032.

Apéndice

A. Comandos de reproducción

Todos los comandos se ejecutan desde el directorio `h1/` del repositorio `mPES`, con el entorno virtual `win_mpes_env` activado, Python 3.12, las dependencias de `utils/config/requirements.txt` y las variables `VIRTUAL_ENV`, `PYTHONIOENCODING=utf-8` y `TF_ENABLE_ONEDNN_OPTS=0` (Tabla A.1). El entrenamiento final lee los hiperparámetros de `inputs/best_params.json`; la arquitectura del DQN recurrente y del Transformer, y el número de episodios de todos los modelos, se leen de `config/CONFIG.py`.

Tabla A.1: Comandos de optimización bayesiana, entrenamiento final y evaluación de cada paquete. `<pkg>` es cualquiera de los cinco ensambles optimizables.

Paquete	Optimización bayesiana	Entrenamiento final	Evaluación
<code>pes_base</code>	—	<code>python -m tabular.pes_base.ext.train_rl</code>	<code>python -m tabular.pes_base</code>
<code>pes_ql</code>	<code>python -m tabular.pes_ql.ext.optimize_rl</code>	<code>python -m tabular.pes_ql.ext.train_rl</code>	<code>python -m tabular.pes_ql</code>
<code>pes_dql</code>	<code>python -m tabular.pes_dql.ext.optimize_rl</code>	<code>python -m tabular.pes_dql.ext.train_rl</code>	<code>python -m tabular.pes_dql</code>
<code>pes_dqn</code>	<code>python -m ml.pes_dqn.ext.optimize_dqn</code>	<code>python -m ml.pes_dqn.ext.train_dqn</code>	<code>python -m ml.pes_dqn</code>
<code>pes_rdqn</code>	<code>python -m ml.pes_rdqn.ext.optimize_rdqn</code>	<code>python -m ml.pes_rdqn.ext.train_rdqn</code>	<code>python -m ml.pes_rdqn</code>
<code>pes_a2c</code>	<code>python -m ml.pes_a2c.ext.optimize_a2c</code>	<code>python -m ml.pes_a2c.ext.train_a2c</code>	<code>python -m ml.pes_a2c</code>
<code>pes_trf</code>	<code>python -m ml.pes_trf.ext.optimize_tr</code>	<code>python -m ml.pes_trf.ext.train_transformer</code>	<code>python -m ml.pes_trf</code>
<code>pes_ens</code>	—	—	<code>python -m ens.pes_ens</code>
<code><pkg></code>	<code>python -m ens.<pkg>.ext.optimize_ens</code>	—	<code>python -m ens.<pkg></code>

B. Ejecución de la evaluación

La evaluación en los 22 escenarios y las 5 réplicas fuera de muestra se ejecuta desde `h1/`:

```
python -m general.scripts.benchmark run --suite both
```

```
python -m general.scripts.benchmark heldout
```

```
python -m general.scripts.analysis
```

```
python -m general.scripts.figures
```

```
python -m general.scripts.random_baseline
```

El primer comando, reanudable, evalúa cada par (modelo, escenario) de los dos grupos reemplazando temporalmente los archivos de entrada de cada paquete por los del escenario. El segundo guarda en `general/results/heldout/` una copia de las réplicas y comprueba que los 13 modelos recibieron las mismas secuencias; cada réplica registra

en `sampling_distribution.json` las frecuencias de sorteo, la semilla y las frecuencias efectivamente obtenidas. El tercero calcula las matrices modelo $\times$ escenario y las comparaciones entre pares; el cuarto genera las figuras, y el quinto, las del agente aleatorio. Los resultados se guardan en `general/results/<grupo>/`.

Las frecuencias de la Tabla 5.4 se obtienen, desde la raíz del repositorio, con `python writings/auxiliar/scripts/ensemble_decisions.py`, que repite la evaluación del ensamble ponderado, del consenso con *prior* y de la compuerta del Transformer, registra en cada decisión la acción antes y después de cada regla y comprueba que las medias coincidan con las de la evaluación principal. La Tabla 5.3 se obtiene con `python writings/auxiliar/scripts/trf_vs_ens.py`, que compara cada ensamble con el Transformer a partir de los resultados guardados en `general/results/`, y la Tabla 5.6, con `python writings/auxiliar/scripts/heldout_gap.py`. Los análisis de la Sección 5.5 se obtienen, desde la misma carpeta, con `weighted_ens_sensitivity.py` (sensibilidad), `weighted_ens_oracle.py` (asignación óptima y efecto de cada regla) y `fixed_rule.py` (regla fija en los 27 escenarios).

C. Mapas estadísticos complementarios

Las figuras de este apéndice detallan, modelo por modelo, las comparaciones estadísticas que el texto informa de forma agregada.

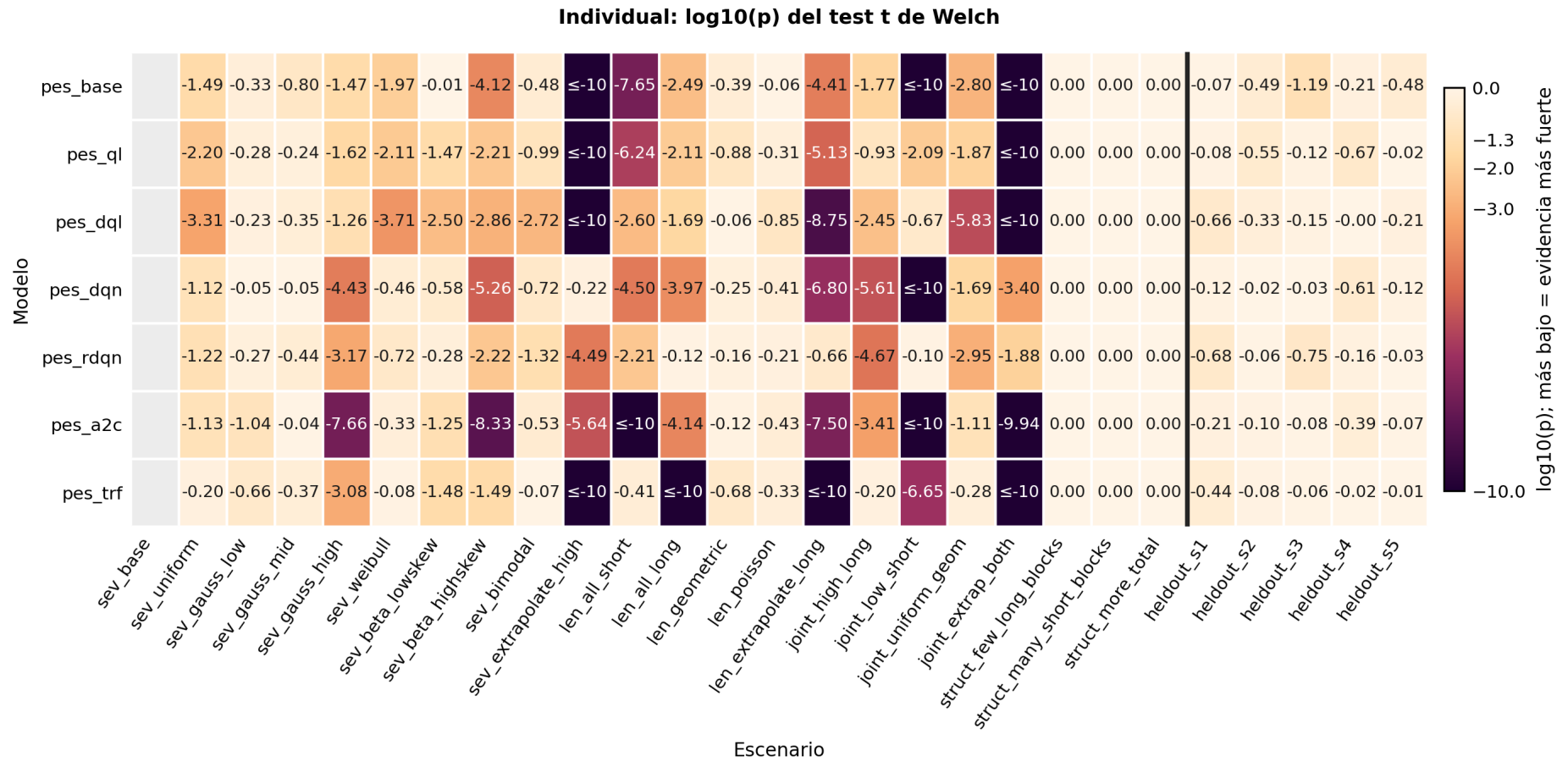


Figura C.1: Modelos individuales: $\log_{10} p$ de Welch de cada escenario frente a la referencia del mismo modelo; los valores menores que -10 se muestran como ≤ -10 . Las cinco columnas a la derecha de la línea vertical son las réplicas fuera de muestra.

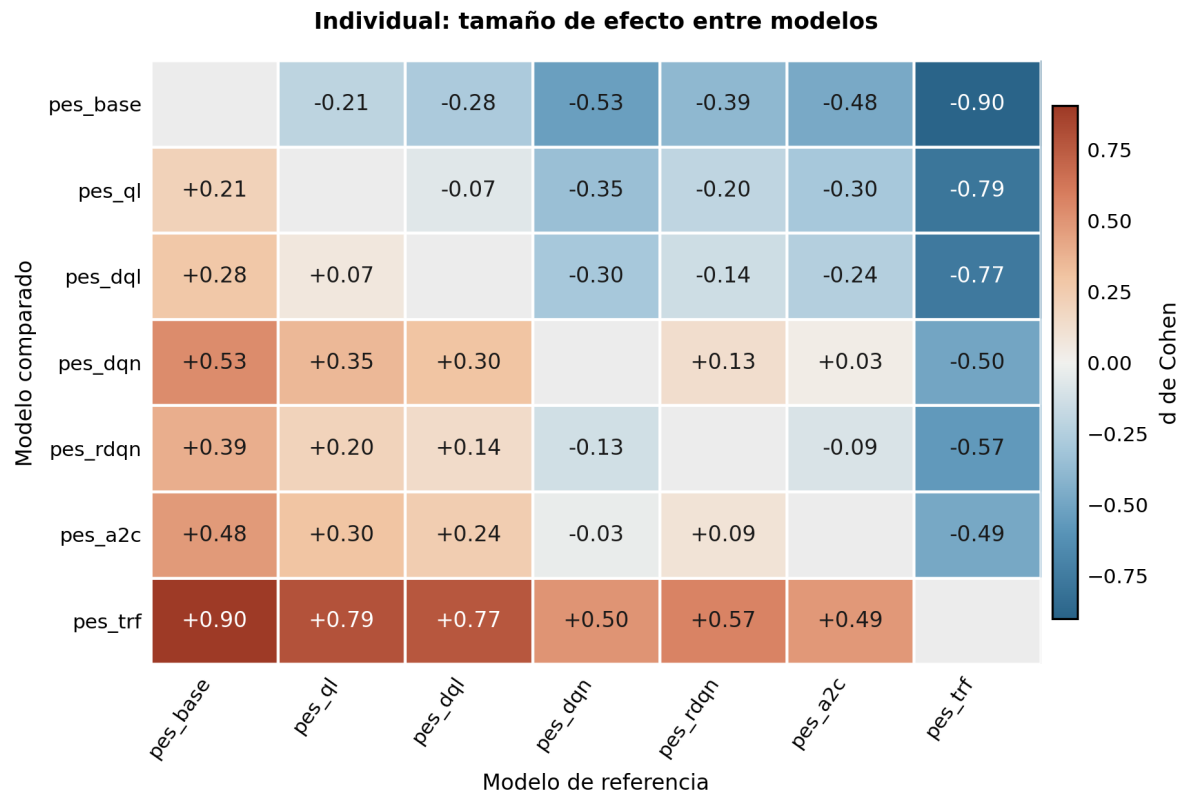


Figura C.2: Modelos individuales: d de Cohen entre pares de modelos en los 21 escenarios de generalización.

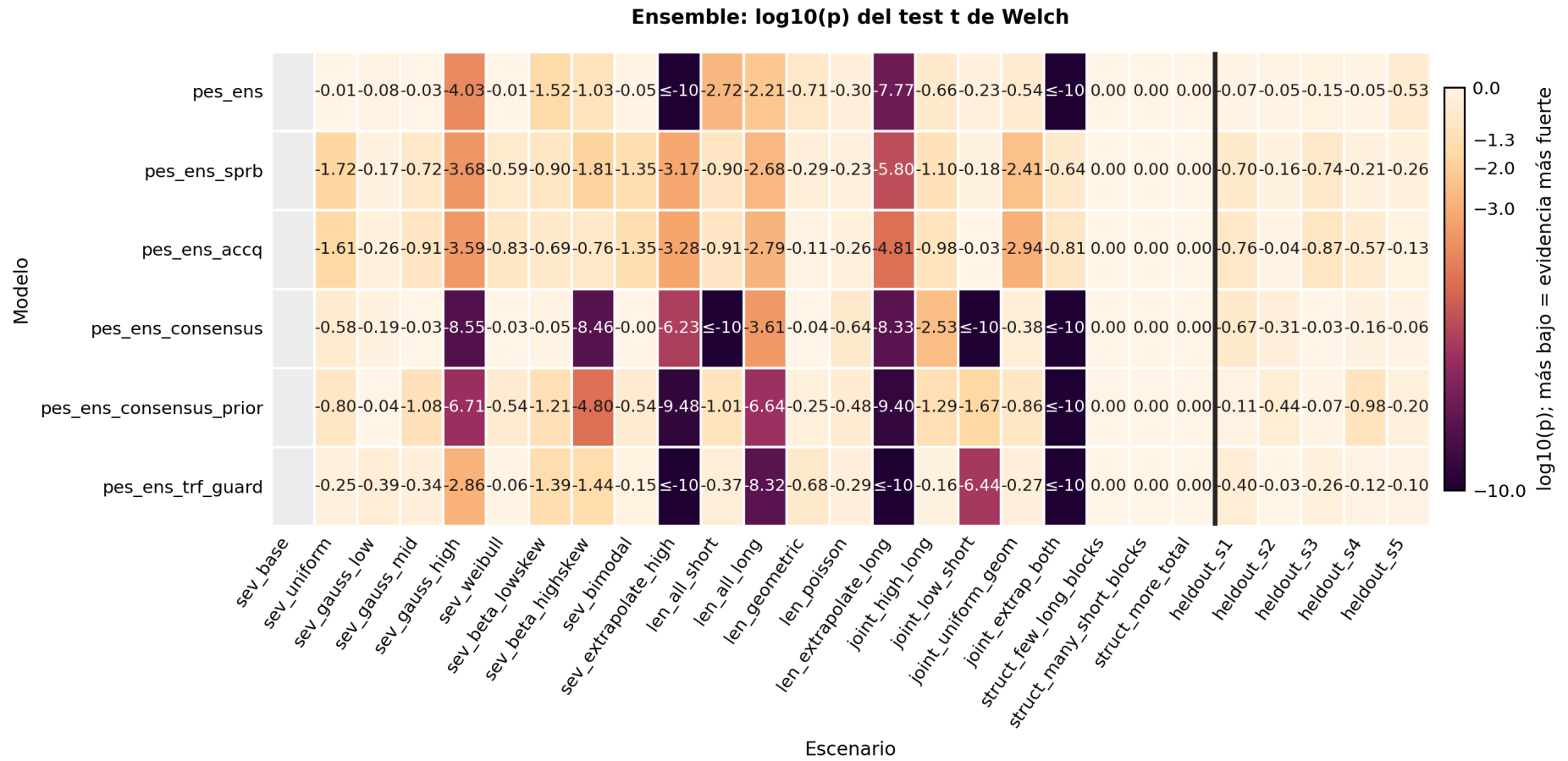


Figura C.3: Ensamblados: log₁₀ p de Welch de cada escenario frente a la referencia del mismo ensamble; los valores menores que -10 se muestran como ≤ -10 . Las cinco columnas a la derecha de la línea vertical son las réplicas fuera de muestra.

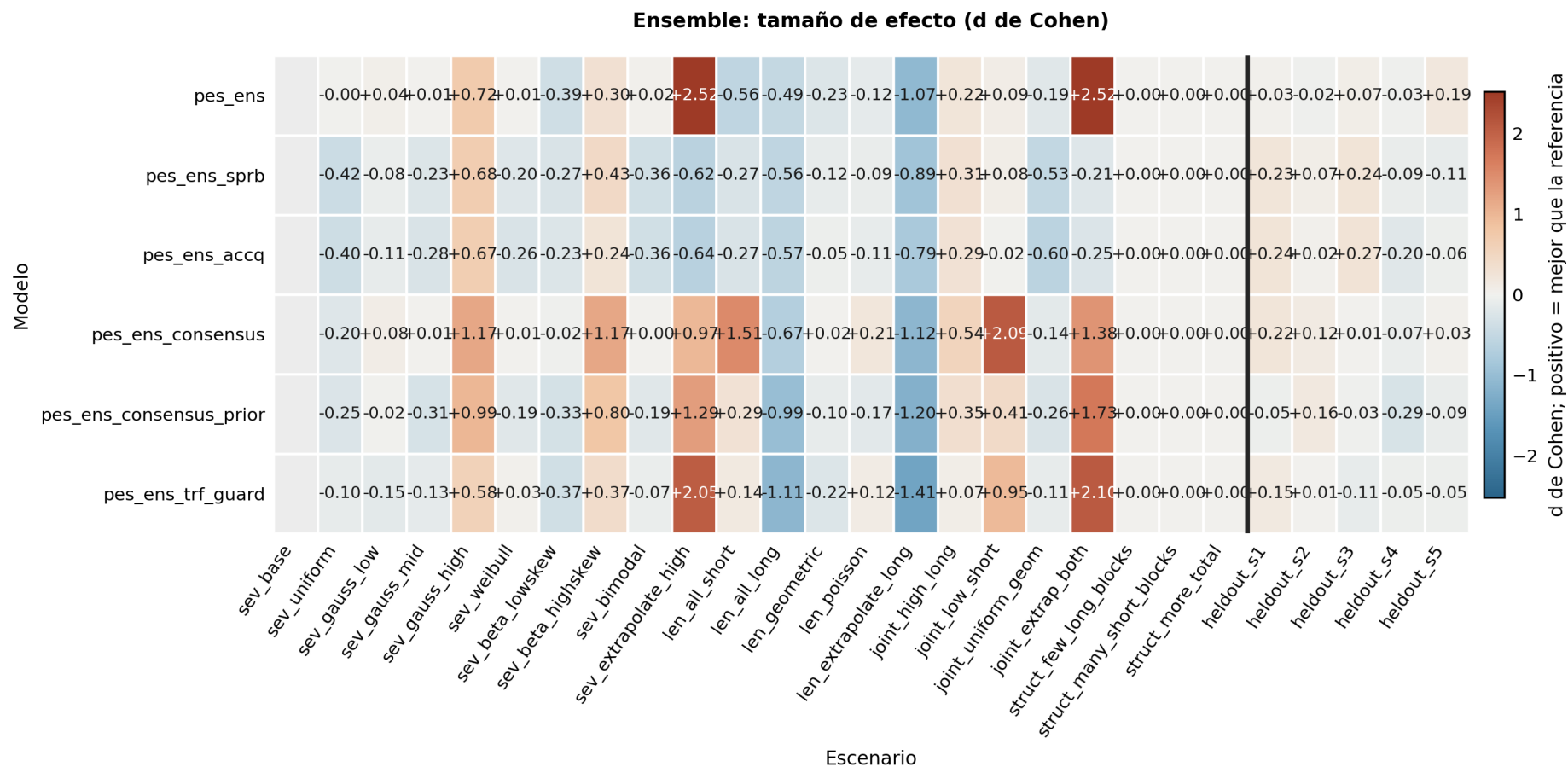
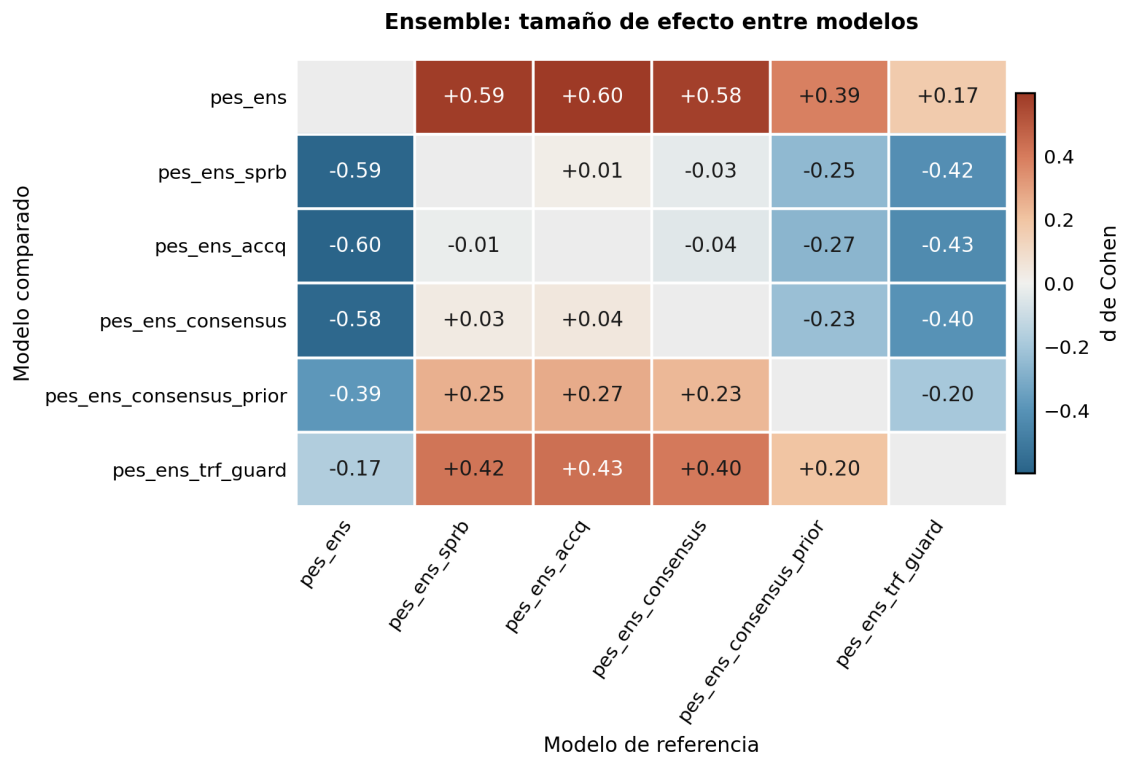
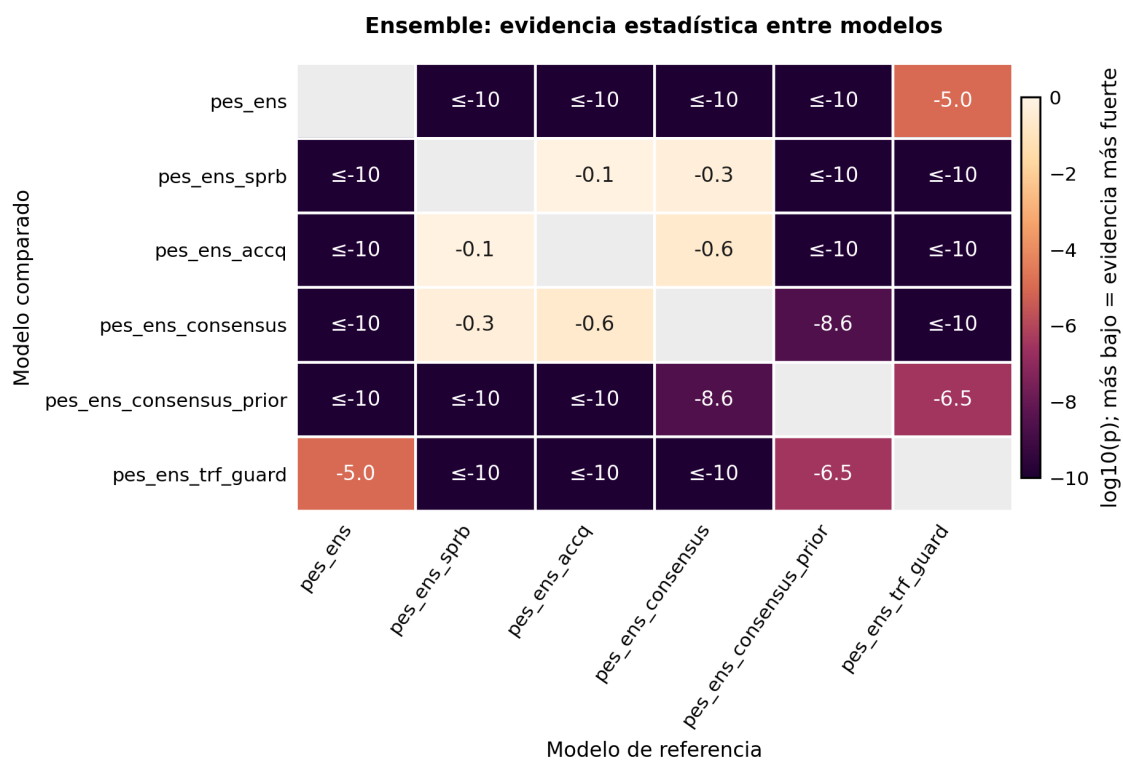


Figura C.4: Ensamblas: d de Cohen de cada escenario frente a la referencia del mismo ensamble (positivo: mejor que la referencia). Las cinco columnas a la derecha de la línea vertical son las réplicas fuera de muestra.

(a) d de Cohen.(b) $\log_{10} p$ de Welch.**Figura C.5:** Ensamblajes: comparación entre pares en los 21 escenarios de generalización.

D. Resultados por secuencia de cada modelo

Las figuras de este apéndice muestran, para cada modelo individual, el desempeño de cada secuencia de la referencia, su distribución y sus estadísticos por bloque (Sección 5.1).

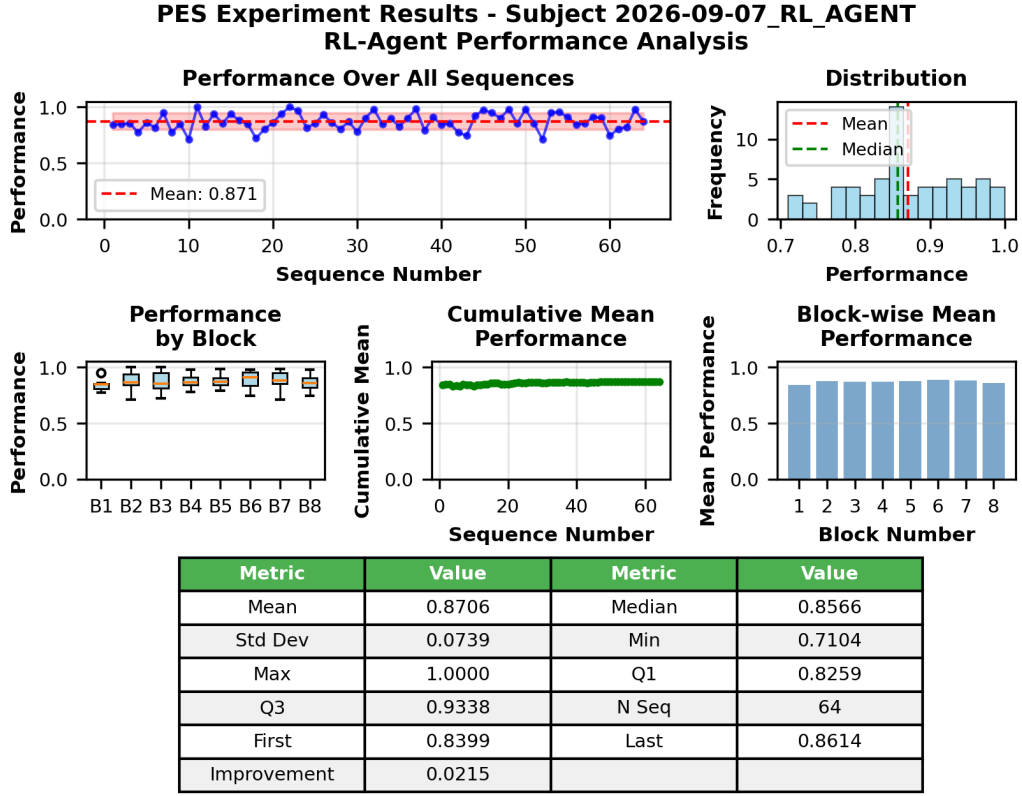


Figura D.1: Resultados por secuencia de Q-Learning base (pes_base) en la referencia.

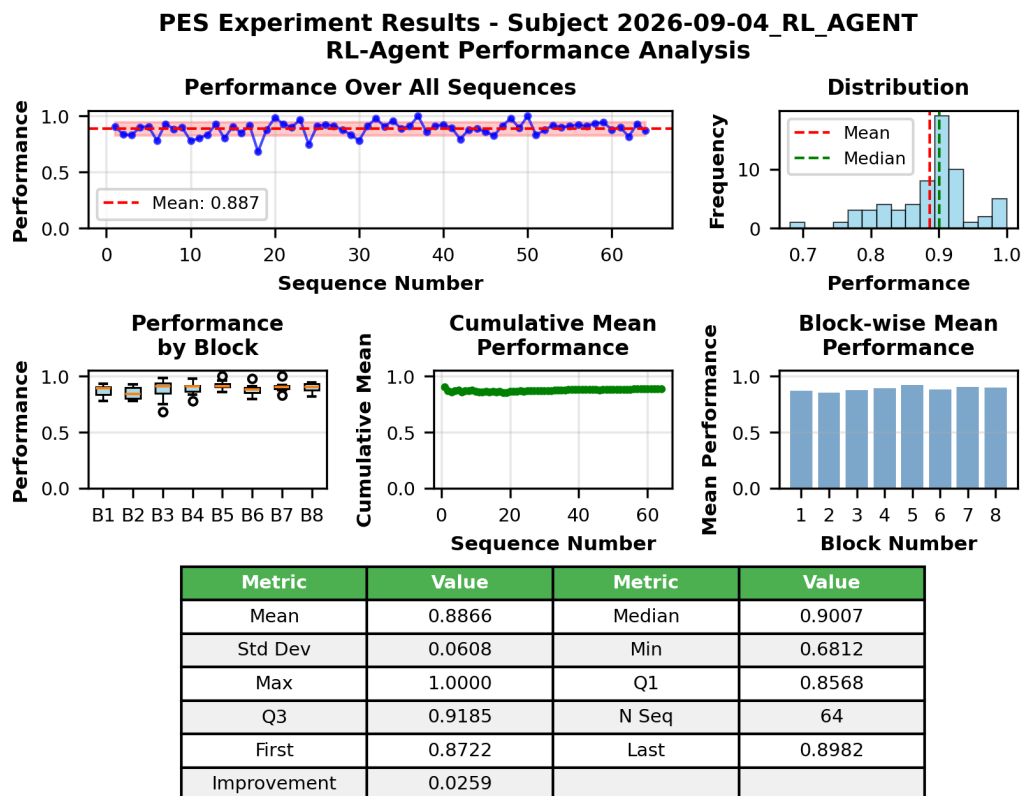


Figura D.2: Resultados por secuencia de Q-Learning (pes_q1) en la referencia.

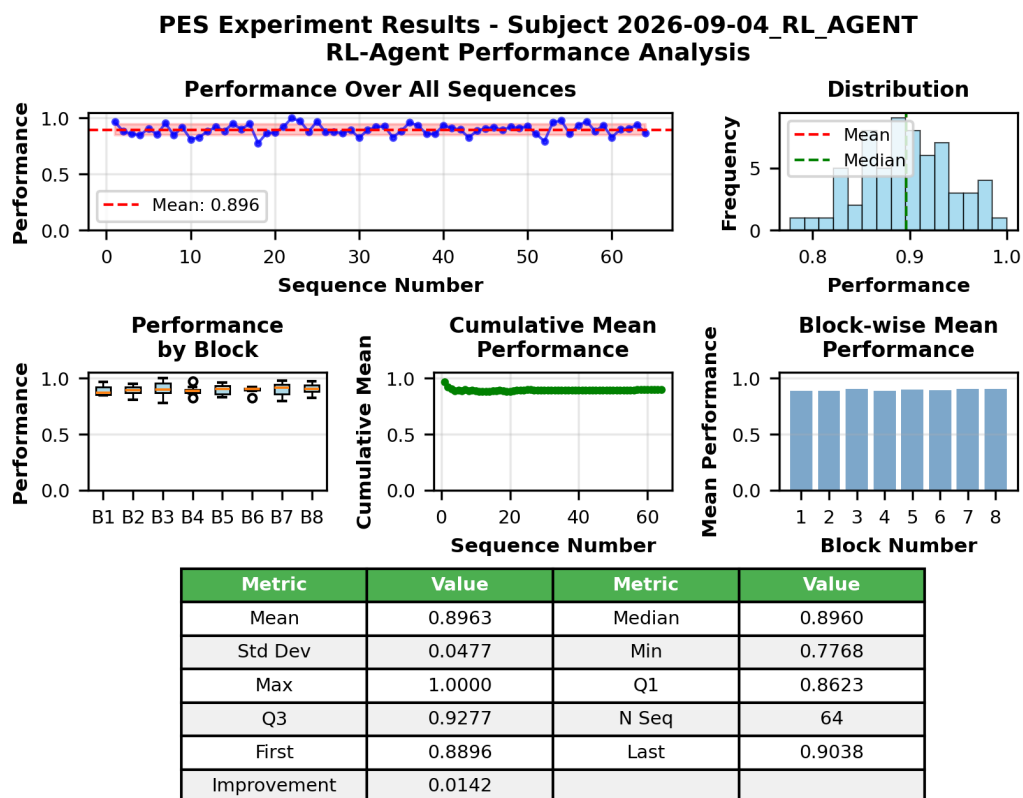


Figura D.3: Resultados por secuencia de Double Q-Learning (pes_dq1) en la referencia.

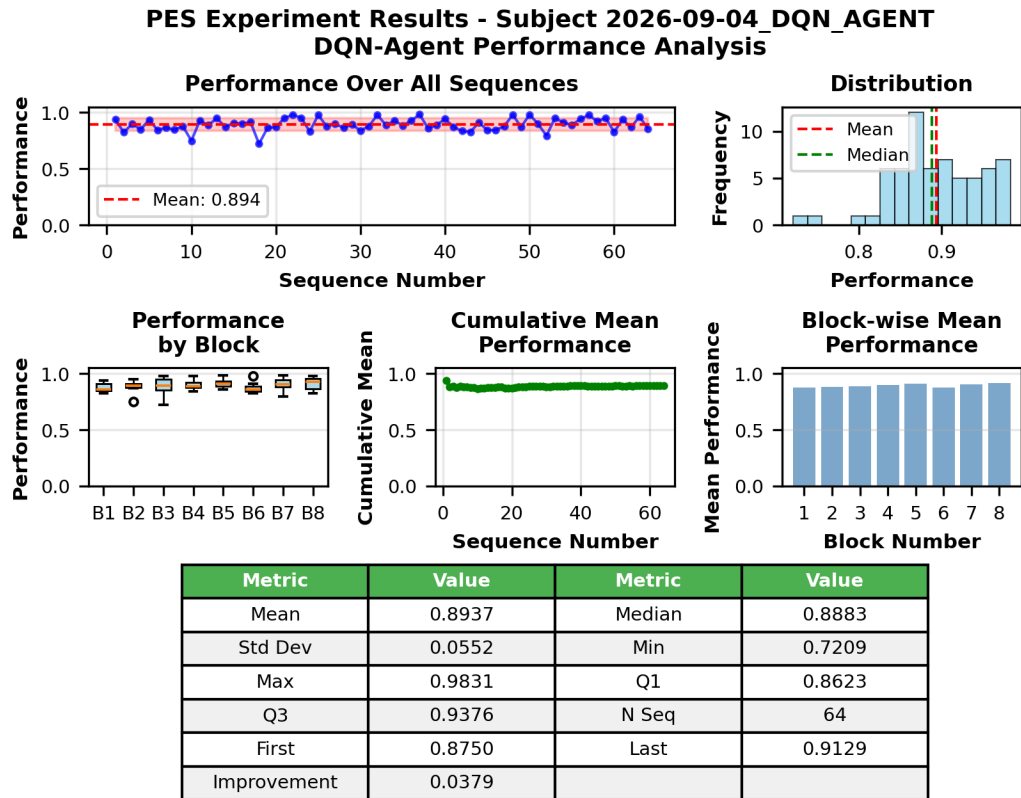


Figura D.4: Resultados por secuencia de DQN (pes_dqn) en la referencia.

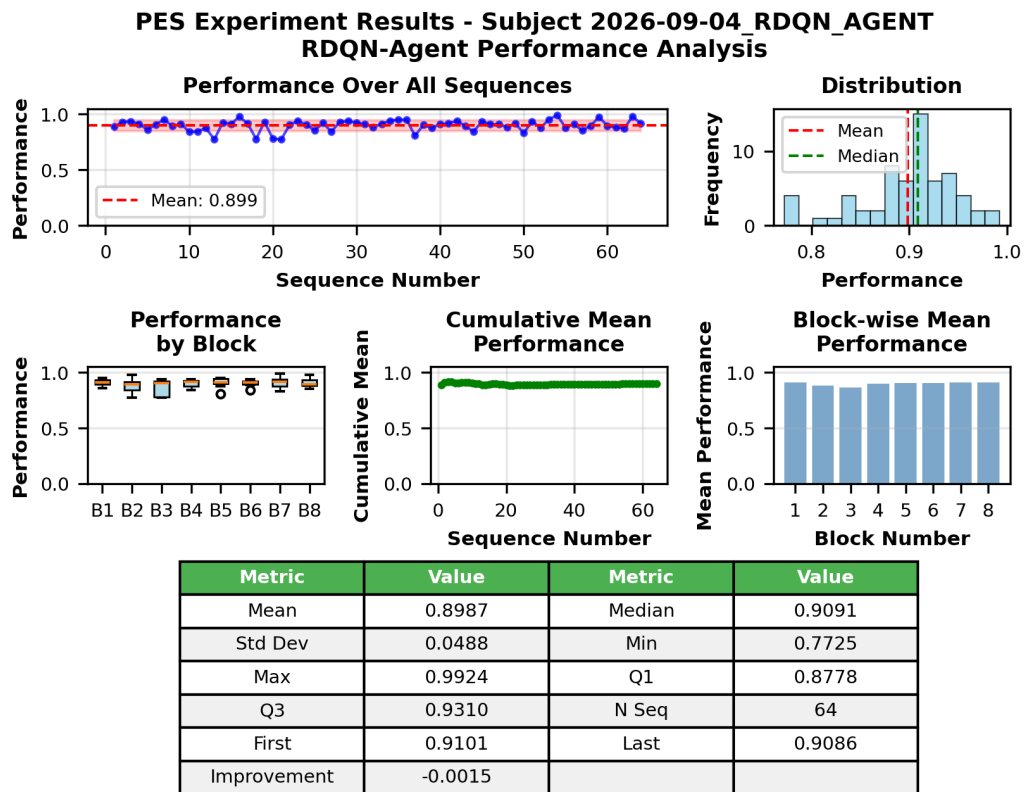


Figura D.5: Resultados por secuencia del DQN recurrente (pes_rdqn) en la referencia.

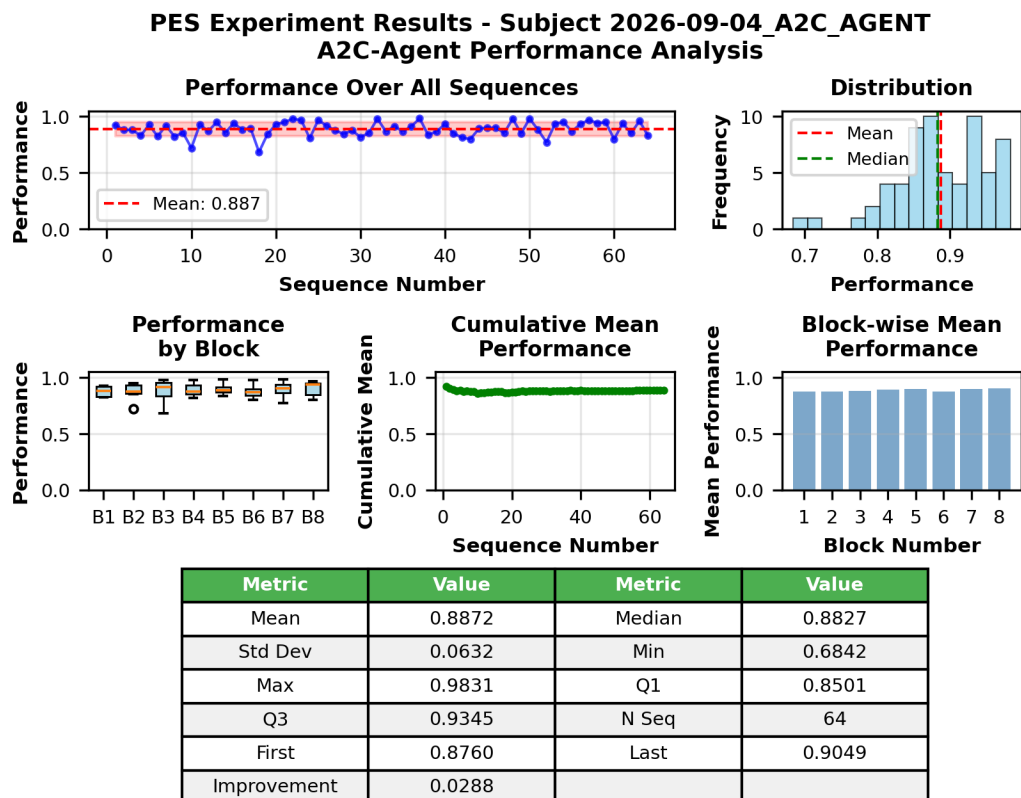


Figura D.6: Resultados por secuencia de A2C (pes_a2c) en la referencia.

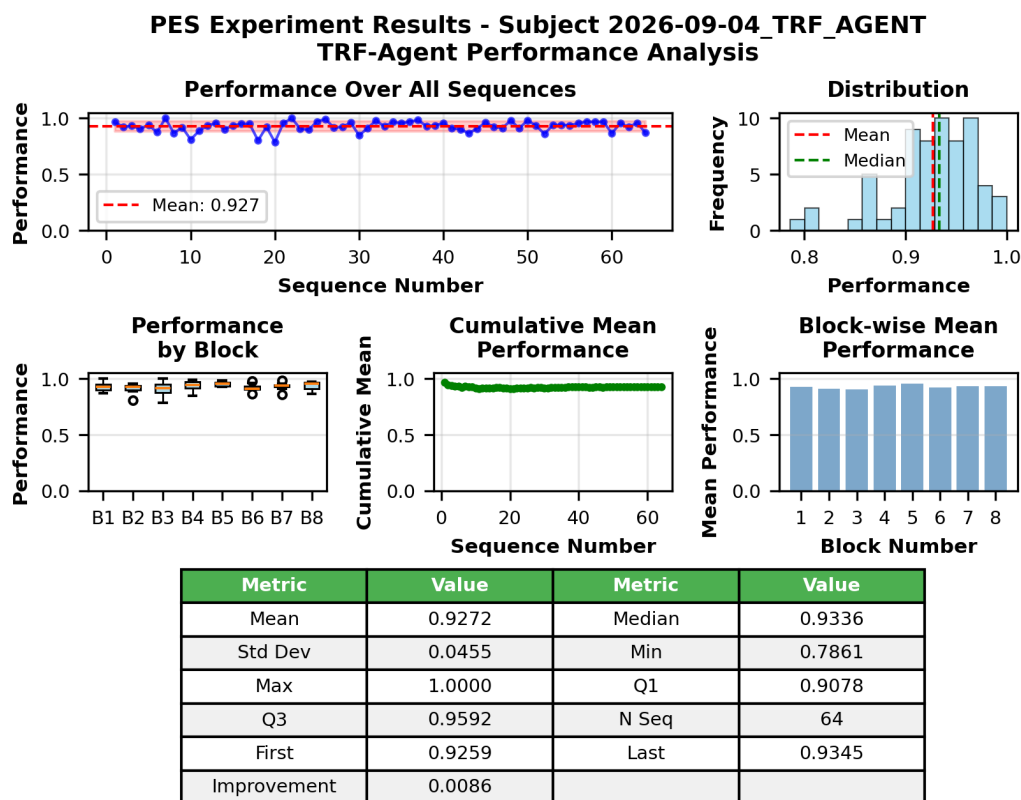


Figura D.7: Resultados por secuencia del Transformer (pes_trf) en la referencia.